Universidad Rey Juan Carlos

Trabajo de Fin de Grado

Enterprise RAG System:

Arquitectura Inteligente de Generación Aumentada por Recuperación con Modelos Locales

Autor:
Ángel María Ballesteros Limones

Tutor:
Jesús María González Barahona

Grado en Ingeniería de Tecnología de Telecomunicaciones
Curso 2025-2026
14 de abril de 2026

Dedicatoria

A mi familia y a mis amigos,
por su apoyo incondicional.

En especial a mi madre y a mi abuelo,
por confiar en mi incluso
cuando yo no lo hacía.

Agradecimientos

Este proyecto no habría sido posible sin el apoyo y la colaboración de numerosas personas e instituciones que han contribuido, directa o indirectamente, a su realización.

En primer lugar, quiero agradecer a mi tutor de TFG por su orientación académica y su disponibilidad a lo largo de todo el proceso de desarrollo, siendo nuestros horarios tan diferentes. Sus revisiones y conocimiento han sido determinantes para conseguir dar forma a la idea que tenía en mente.

Agradezco también a la empresa Europavia por facilitar el entorno real de despliegue, con una necesidad existente a nivel corporativo que ha servido como banco de pruebas del sistema. La oportunidad de desarrollar un proyecto con impacto práctico inmediato en un contexto profesional ha enriquecido enormemente la experiencia. Sobretodo Nacho y CIVEX por ser siempre de ayuda y facilitar cualquier desarrollo que condsiderara.

A mis compañeros de carrera, sobretodo a los de Rugby y a aquellos del Erasmus, que me enseñaron el verdadero sentido de la universidad. El camino se hace más llevadero cuando se recorre en buena compañía.

A mi familia, por su paciencia infinita durante los meses más intensos de exámenes. Su apoyo incondicional ha sido el pilar fundamental sobre el que se sustenta todo lo demás.

Por último, quiero reconocer a la comunidad de código abierto, cuyos proyectos — Ollama, Qdrant, FastAPI, OpenWebUI, Playwright, PaddleOCR, LiteLLM, entre muchos otros — han hecho posible construir un sistema de esta envergadura sin dependencias comerciales. Este TFG es, en gran medida, un tributo a la filosofía del software libre.

Ángel María Ballesteros Limones
Madrid, 2026

Resumen

Este Trabajo de Fin de Grado presenta el diseño, desarrollo y despliegue de un sistema de Generación Aumentada mediante Recuperación (*Retrieval-Augmented Generation*, RAG) de ámbito corporativo, ejecutado íntegramente en infraestructura local (*on-premise*) y basado en la integración de modelos y herramientas existentes de código abierto junto con desarrollo propio.

El sistema permite a los empleados de una organización interactuar con sus documentos internos, páginas web y fuentes legislativas oficiales mediante una interfaz conversacional basada en lenguaje natural. La arquitectura emplea una pila de microservicios contenerizados con Docker que integra modelos de lenguaje ejecutados localmente mediante Ollama, una base de datos vectorial Qdrant para la búsqueda semántica y un agente encaminador denominado JARVIS, que detecta automáticamente la intención del usuario y delega cada consulta al modo de operación más adecuado.

Entre las fuentes de información integradas se incluyen Microsoft SharePoint, que ha sido la fuente principal de validación práctica del sistema; páginas web procesadas mediante scraping y extracción de contenido; documentos PDF escaneados tratados con OCR acelerado por GPU; y el Boletín Oficial del Estado (BOE), integrado funcionalmente por API y complementado con un servidor MCP desarrollado como línea de evolución futura.

El sistema ha sido desplegado y validado en un entorno real con resultados satisfactorios, alcanzando tiempos de respuesta inferiores a 3.5 segundos y reduciendo de forma muy significativa las alucinaciones en modo RAG mediante prompts restrictivos, baja temperatura de inferencia y una estrategia de recuperación documental diseñada para priorizar la trazabilidad.

Palabras clave: RAG, LLM, inteligencia artificial, embeddings, base de datos vectorial, microservicios, Docker, Ollama, Qdrant, on-premise, código abierto.

Abstract

This Bachelor's Thesis presents the design, development and deployment of a corporate Retrieval-Augmented Generation (RAG) system, executed entirely on local infrastructure (on-premise) and based on the integration of existing open-source models and tools together with original development.

The system enables employees of an organization to interact with internal documents, web pages and official legislative sources through a natural-language conversational interface. The architecture relies on containerized microservices orchestrated with Docker, integrating locally executed language models through Ollama, a Qdrant vector database for semantic search, and a routing agent called JARVIS that automatically detects user intent and delegates each query to the most suitable operating mode.

Integrated information sources include Microsoft SharePoint, which served as the main real-world validation source; web pages processed through scraping and content extraction; scanned PDF documents handled through GPU-accelerated OCR; and the Spanish Official State Gazette (BOE), functionally integrated through API calls and complemented by an MCP server developed as a future integration path.

The system has been deployed and validated in a real environment with satisfactory results, achieving response times under 3.5 seconds and significantly reducing hallucinations in RAG mode through restrictive prompting, low-temperature inference and a retrieval strategy focused on traceable evidence.

Keywords: RAG, LLM, artificial intelligence, embeddings, vector database, microservices, Docker, Ollama, Qdrant, on-premise, open source.

Índice general

Agradecimientos **v**

Resumen **vii**

Abstract **ix**

1. Introducción **1**

1.1. Motivación 2

1.1.1. Problemática del Shadow AI en entornos corporativos 3

1.2. Solución propuesta y enfoque del trabajo 4

1.3. Código del proyecto 5

1.4. Objetivos del proyecto 5

1.5. Alcance y limitaciones 6

1.5.1. Alcance 6

1.5.2. Limitaciones 7

1.6. Contribuciones principales 8

1.7. Estructura de la memoria 9

2. Fundamentos de la solución propuesta **11**

2.1. Grandes Modelos de Lenguaje (LLMs) 11

2.1.1. Propietarios vs. pesos abiertos 12

2.2. Por qué un LLM por sí solo no resuelve este problema 12

2.3. Generación Aumentada mediante Recuperación (RAG) 13

2.3.1. Mejoras prácticas: búsqueda híbrida y memoria acotada 13

2.4. Modelos de Embeddings 14

2.5. Bases de datos vectoriales 14

2.6. Cuantización y ejecución local 15

2.7. Ajuste fino eficiente con LoRA 15

2.8. Model Context Protocol (MCP) 16

3. Tecnologas utilizadas y criterio de eleccin 17
3.1. Python como lenguaje de integracin principal 18
3.2. OpenWebUI: interfaz conversacional reutilizada 19
3.2.1. Arquitectura interna de OpenWebUI 19
3.2.2. Por qu OpenWebUI 20
3.2.3. Sistema de Pipelines de OpenWebUI 20
3.3. Ollama: ejecucin local de modelos 21
3.3.1. Caractersticas tcnicas 21
3.3.2. Modelos desplegados 21
3.4. Qdrant: base de datos vectorial 22
3.4.1. Caractersticas utilizadas 22
3.5. FastAPI: backend del sistema 22
3.6. LiteLLM: proxy unificado de modelos 23
3.7. Docker y orquestacin de contenedores 23
3.7.1. Ventajas de la contenerizacin 23
3.8. Herramientas de procesamiento documental y web 23
3.8.1. PaddleOCR: reconocimiento ptico de caracteres 24
3.8.2. Playwright: automatizacin de navegadores 24
3.8.3. Trafilatura: extraccin de contenido web 24
3.8.4. SentenceTransformers: generacin de embeddings 24
3.9. Stack de monitorizacin 24
3.9.1. Prometheus 25
3.9.2. Grafana 25
3.9.3. NVIDIA DCGM Exporter 25
3.10. Otras tecnologas del ecosistema 25
3.11. Diagrama de flujo de tecnologas usadas 26

4. Sistemas relacionados 29
4.1. Asistentes comerciales basados en LLM 29
4.1.1. ChatGPT y GPT Enterprise (OpenAI) 29
4.1.2. Microsoft Copilot para Microsoft 365 30
4.1.3. Google Gemini y Vertex AI 30
4.1.4. Amazon Bedrock 30
4.2. Herramientas abiertas 30
4.2.1. PrivateGPT 30
4.2.2. AnythingLLM 31
4.2.3. Danswer / Onyx 31
4.2.4. OpenClaw y la aceleración reciente del ecosistema 31

4.2.5. NotebookLM y asistentes centrados en conocimiento aportado por el usuario . . . 32
4.2.6. Frameworks: LangChain y LlamaIndex . . . 32
4.3. Análisis comparativo . . . 32
4.4. Posicionamiento del sistema desarrollado . . . 33

5. Descripción Funcional del Sistema 35
5.1. Visión General . . . 35
5.2. Acceso al Sistema e Inicio de Sesión . . . 36
5.3. Interfaz de Usuario . . . 36
5.3.1. Panel Principal de Chat . . . 37
5.3.2. Panel Lateral de Conversaciones . . . 37
5.3.3. Carga de Archivos Adjuntos . . . 37
5.4. Modos de Operación . . . 38
5.4.1. Consulta de Documentos Internos (RAG) . . . 38
5.4.2. Búsqueda en Internet . . . 38
5.4.3. Análisis de Páginas Web (Scraping) . . . 39
5.4.4. Análisis de Imágenes (Visión/OCR) . . . 39
5.4.5. Consulta Legislativa del BOE . . . 40
5.4.6. Listar Documentos Disponibles . . . 40
5.4.7. Estado del Sistema . . . 40
5.4.8. Ayuda y Capacidades . . . 41
5.4.9. Conversación General . . . 41
5.5. Escenarios de Uso Completos . . . 41
5.5.1. Escenario 1: Consulta de un procedimiento corporativo . . . 41
5.5.2. Escenario 2: Análisis de una normativa del BOE . . . 42
5.5.3. Escenario 3: Análisis de una web tecnológica . . . 42
5.6. Panel de Administración y Monitorización . . . 42
5.6.1. Dashboards de Grafana . . . 42
5.6.2. Administración de la Base de Datos . . . 43
5.6.3. Gestión de Modelos . . . 43

6. Diseño e Implementación 45
6.1. Filosofía de Diseño: Necesidad, Solución, Iteración . . . 45
6.2. Arquitectura General del Sistema . . . 47
6.2.1. Orquestación con Docker Compose . . . 47
6.2.2. Flujo de Datos End-to-End . . . 49
6.3. Desarrollo Propio vs. Herramientas Externas . . . 51

6.4. Pipeline de OpenWebUI 53
6.4.1. Concepto de Hook 53
6.4.2. Descubrimiento y Conexión del Pipeline 53
6.4.3. Contrato de la Clase Pipeline 54
6.4.4. El Método `pipe()`: Corazón del Sistema 54
6.4.5. Sistema Valves: Configuración Dinámica 56
6.5. Agente Encaminador JARVIS: Inteligencia Central 56
6.5.1. Sistema de Detección de Intenciones 57
6.5.2. Sticky Mode: Persistencia de Contexto 59
6.5.3. Memoria de Sesión por Usuario 59
6.5.4. Control de Acceso Departamental 60
6.6. Backend RAG: Motor de Búsqueda Semántica 60
6.6.1. Pipeline de Ingesta Documental 61
6.6.2. Búsqueda Híbrida 61
6.6.3. Construcción del Prompt Contextual 62
6.7. Extracción Web: Scraping y Búsqueda en Internet 62
6.7.1. Motor de Renderizado con Playwright 62
6.7.2. Cadena de Extracción con Fallback 63
6.7.3. Búsqueda en Internet 63
6.8. Reconocimiento Óptico de Caracteres (OCR) 63
6.9. Integración con Microsoft SharePoint 64
6.9.1. Autenticación MSAL 64
6.9.2. Sincronización Delta Incremental 64
6.10. Integración con el Boletín Oficial del Estado (BOE) 64
6.10.1. Resolución Semántica de Leyes 65
6.11. Seguridad Perimetral y Autenticación 66
6.11.1. NGINX como Proxy Inverso 66
6.11.2. SSO con Azure Active Directory 66
6.11.3. Validación JWT en el Backend 66
6.11.4. Aislamiento de Red 67
6.12. Stack de Observabilidad: Prometheus y Grafana 67
6.12.1. Métricas Personalizadas en el Backend 67
6.12.2. Telemetría GPU con NVIDIA DCGM Exporter 68
6.12.3. Diseño de Dashboards en Grafana 68
6.12.4. Impacto de la Monitorización 69
6.13. Caché y Rendimiento: Redis 69
6.14. Almacenamiento de Objetos: MinIO 70

6.15. Agente SQL: Consultas en Lenguaje Natural sobre Datos Estructurados 70
6.15.1. Arquitectura del Agente NL→SQL 70
6.15.2. Mecanismos de Seguridad 71
6.15.3. Endpoint Unificado RAG+SQL 71
6.16. Fine-Tuning con LoRA 72
6.16.1. Generación del Dataset de Entrenamiento 72
6.16.2. Entrenamiento con Low-Rank Adaptation 72
6.16.3. Resultado y Evaluación 72

7. Metodología 73
7.1. Proceso de Desarrollo 73
7.1.1. Iteraciones de Desarrollo 74
7.1.2. Diagrama de GANTT 75
7.2. Entorno Hardware 75
7.3. Stack Tecnológico 76
7.4. Herramientas y Control de Versiones 77
7.4.1. Integración Continua con GitHub Actions 77
7.5. Estimación de Presupuesto 77
7.6. Gestión de Riesgos 78
7.7. Impacto de Asignaturas Cursadas en la Carrera 78
7.7.1. Conocimientos Aplicados del Grado 78
7.7.2. Conocimientos Adquiridos de Forma Autónoma 79
7.8. Estrategia de Documentación 80
7.8.1. Redacción de la Memoria 80

8. Resultados y Evaluación 81
8.1. Métricas de Rendimiento 81
8.1.1. Análisis de Latencia 82
8.1.2. Gestión de VRAM 83
8.2. Evaluación Funcional 83
8.2.1. Consulta RAG sobre Documentos Corporativos 83
8.2.2. Scraping de URL 84
8.2.3. Búsqueda Legal en el BOE 84
8.2.4. Análisis de Imagen 84
8.2.5. Consultas NL→SQL con el Agente SQL 84
8.2.6. Encaminamiento Inteligente 85
8.3. Mitigación de Alucinaciones 86
8.4. Impacto de la Caché 86

9. Conclusiones y Trabajo Futuro **87**
9.1. Resumen General 87
9.2. Consecución de Objetivos 87
9.2.1. Funcionalidades logradas 88
9.3. Comparación con Otros Sistemas 89
9.4. Código del proyecto 89
9.5. Lecciones Aprendidas 89
9.6. Estimación de Dedicación Total 90
9.7. Trabajo Futuro y Líneas de Expansión 90
9.8. Impacto y Transferencia 91

Bibliografía **93**

Índice de figuras

3.1. Diagrama general de las tecnologas utilizadas en el sistema. La disposicin apaisada separa el flujo de consulta, la lgica propia del TFG, las integraciones externas y la capa de observabilidad en bloques legibles. 27

6.1. Interfaz de OpenWebUI con el modelo JARVIS seleccionado. El usuario escribe su consulta en el campo de texto y recibe la respuesta en streaming. 50

6.2. Ejemplo de respuesta RAG con fuentes documentales. El sistema muestra el contenido recuperado junto con las referencias a los documentos y páginas de origen. 59

6.3. Dashboard de Grafana mostrando las métricas del backend en tiempo real: latencia, distribución de modos de operación y tasa de errores. . 69

7.1. Diagrama de GANTT del proyecto. 75

8.1. Descomposición de la latencia por componente para una consulta RAG típica. La inferencia del LLM domina el tiempo total del flujo. . 82

Índice de tablas

2.1. Comparativa de bases de datos vectoriales evaluadas para el proyecto. 14

3.1. Comparativa de interfaces de usuario para LLMs locales. 20
3.2. Modelos de IA desplegados en Ollama y su funcin en el sistema. . . . 21

4.1. Comparación del sistema JARVIS con soluciones alternativas. 32

6.1. Servicios principales de la arquitectura y su función. 47
6.2. Resumen sintético de reutilización y desarrollo dentro del TFG. . . . 51
6.3. Clasificación de los componentes del sistema según su origen. 52
6.4. Categorías de intención y su prioridad de evaluación. 58

7.1. Iteraciones principales del desarrollo, entregables y horas dedicadas. . 74
7.2. Especificaciones del entorno hardware de despliegue. 75
7.3. Stack tecnológico completo del proyecto. 76
7.4. Estimación de presupuesto para el despliegue del sistema. 77
7.5. Riesgos técnicos identificados y estrategias de mitigación. 78

8.1. Métricas de rendimiento del sistema en producción. 81
8.2. Desglose de latencia por componente para una consulta RAG típica. . 82
8.3. Consumo de VRAM por modelo y combinaciones de carga simultánea. 83
8.4. Resultados de la evaluación del encaminamiento de JARVIS. 85

9.1. Checklist de funcionalidades logradas. 88
9.2. Distribución de horas dedicadas al proyecto. 90

Capítulo 1

Introducción

1.1 Motivación

En los entornos corporativos actuales, el volumen de información disponible crece de forma constante, pero no crece de forma ordenada. La documentación relevante suele estar repartida entre repositorios como Microsoft SharePoint, PDFs internos, muchos de ellos escaneados, wikis, portales de proveedores, procedimientos técnicos y fuentes públicas como el Boletín Oficial del Estado (BOE). En la práctica, el problema habitual no es la falta de información, sino la dispersión, la heterogeneidad de formatos y la dificultad para recuperar contenido útil cuando se necesita.

Los métodos tradicionales de búsqueda por palabras clave fallan en situaciones muy comunes: cuando el usuario no conoce el término exacto, cuando la información está contenida en documentos largos, cuando existen sinónimos o acrónimos internos, o cuando el contenido está en PDFs escaneados y requiere OCR. Como consecuencia, tareas rutinarias como confirmar un procedimiento, localizar un requisito normativo o recuperar una especificación se convierten en procesos lentos y propensos a errores.

En paralelo, la popularización de los grandes modelos de lenguaje (LLMs, *Large Language Models*) ha permitido interactuar con sistemas de información mediante lenguaje natural. Sin embargo, en un contexto corporativo, una respuesta fluida pero incorrecta tiene un coste elevado: puede inducir decisiones erróneas y generar confianza injustificada. El problema se agrava cuando el modelo no dispone de acceso al conocimiento interno, cuando la documentación cambia con frecuencia o cuando la consulta requiere precisión y respaldo documental.

Por este motivo, resulta especialmente relevante el enfoque de Generación Aumentada mediante Recuperación (RAG, *Retrieval-Augmented Generation*), que combina la generación de lenguaje con la recuperación de información desde una base documental. En lugar de responder de memoria, el sistema fundamenta la respuesta en documentos recuperados, mejorando la utilidad, la trazabilidad y reduciendo la probabilidad de respuestas no justificadas [1].

Este Trabajo de Fin de Grado no parte de entrenar un modelo desde cero. Sería un objetivo desproporcionado para el alcance del proyecto y, además, no resolvería por sí mismo el problema principal, que no es crear otro LLM generalista, sino poner modelos ya existentes a trabajar de forma útil, segura y controlada sobre información corporativa cambiante. Por ello, el trabajo se centra en seleccionar, integrar, adaptar y orquestar tecnologías existentes de código abierto y pesos abiertos: modelos de lenguaje ejecutados localmente, embeddings, una base de datos vectorial, un backend propio en Python, mecanismos de ingesta documental y una interfaz conversacional empresarial.

Con esta motivación, el proyecto aborda el diseño, desarrollo e integración de un sistema RAG corporativo desplegado en infraestructura controlada (*on-premise*), capaz de ingerir documentos desde fuentes heterogéneas y ofrecer a los usuarios una interfaz conversacional orientada a consultas técnicas, con recuperación semántica y soporte documental.

1.1.1 Problemática del Shadow AI en entornos corporativos

La adopción de asistentes de IA comerciales en el ámbito laboral ha impulsado un fenómeno cada vez más habitual: el uso no autorizado de herramientas externas por parte de empleados sin supervisión del departamento de TI ni revisión legal o de cumplimiento. Este patrón se conoce como *Shadow AI* y puede considerarse una extensión del *Shadow IT*, con un riesgo añadido: la exposición de datos puede producirse en el mismo momento en que un usuario copia y pega información interna en un servicio externo [2].

La magnitud del fenómeno es considerable. Según el informe *Work Trend Index* de Microsoft y LinkedIn (2024), el 75 % de los trabajadores del conocimiento utiliza herramientas de IA en su trabajo, y el 78 % de ellos emplea herramientas propias no autorizadas por la organización [2]. En un análisis de Cyberhaven sobre 1,6 millones de trabajadores, se detectó que el 8,6 % de los empleados introducía datos corporativos en ChatGPT, y que el 11 % de esos datos estaba clasificado como confidencial [3]. Además, entre marzo de 2023 y marzo de 2024, la cantidad de datos corporativos introducidos en herramientas de IA se multiplicó por 4,85, y la proporción de datos sensibles casi se triplicó, pasando del 10,7 % al 27,4 % [4].

Los riesgos principales asociados a este fenómeno son:

- *Exposición de información sensible*: al introducir documentos, fragmentos de procedimientos, incidencias, datos operativos o contenido contractual en herramientas externas, la información puede salir del perímetro de seguridad corporativo. Un estudio de Komprise (2025) reveló que el 44 % de las organizaciones ha experimentado fugas de datos sensibles hacia herramientas de IA [5].

- *Riesgo regulatorio y de cumplimiento*: en sectores regulados o con datos personales o sensibles, el tratamiento de la información exige garantías sobre finalidad, acceso, ubicación y medidas de seguridad. El uso informal de herramientas externas introduce incertidumbre y eleva el riesgo de incumplimiento normativo.

- *Calidad y consistencia*: los modelos generalistas pueden producir respuestas plausibles incluso cuando carecen de base documental. En ámbitos técnicos o normativos, esto se traduce en errores silenciosos con apariencia de certeza.

- *Ausencia de gobernanza*: sin control centralizado, la organización no sabe qué herramientas se usan, con qué datos, con qué finalidad ni con qué impacto.

Este proyecto aborda directamente esta problemática mediante una alternativa interna: un sistema conversacional desplegado en la infraestructura de la organización, donde los documentos no se envían a servicios externos y donde las respuestas se apoyan en recuperación documental, reduciendo respuestas no fundamentadas y permitiendo trazabilidad hacia fuentes internas.

1.2 Solución propuesta y enfoque del trabajo

La solución propuesta consiste en construir un asistente corporativo que actúe como punto único de acceso a la información de la organización. La idea no es sustituir todos los sistemas documentales existentes, sino superponer una capa de consulta en lenguaje natural que permita a cualquier usuario localizar y comprender información dispersa sin necesidad de conocer de antemano dónde está almacenada ni en qué formato se encuentra.

Para resolver los problemas descritos en la sección anterior, el sistema combina varios bloques:

- *Modelos ya existentes ejecutados en local*: se reutilizan LLMs y modelos de embeddings disponibles públicamente, desplegados dentro de la infraestructura propia.

- *Un backend propio en Python*: se desarrolla la lógica que conecta los modelos con las fuentes documentales, aplica reglas de acceso y construye respuestas trazables.

- *Un pipeline RAG*: antes de responder, el sistema recupera fragmentos relevantes de la documentación para que la generación se apoye en evidencia.

- *Conectores e integraciones*: se incorporan mecanismos para trabajar con SharePoint, documentos escaneados, páginas web y normativa del BOE.

- *Una interfaz conversacional reutilizable*: en lugar de crear una interfaz desde cero, se aprovecha OpenWebUI y se amplía mediante un pipeline propio.

Este enfoque tiene dos ventajas. La primera es práctica: permite construir un sistema funcional en el contexto de un TFG sin depender del entrenamiento masivo de modelos. La segunda es metodológica: separa claramente qué aporta la tecnología base y qué aporta el trabajo realizado en el proyecto. En este caso, la contribución no está en crear un nuevo modelo fundacional, sino en diseñar una arquitectura útil, explicar por qué cada pieza es necesaria y desarrollar la lógica que convierte herramientas sueltas en un sistema corporativo coherente.

1.3 Código del proyecto

Tanto el código fuente como la documentación técnica del proyecto están publicados en GitHub[1]. Este repositorio incluye la definición de la arquitectura Docker, los servicios implementados en Python, la configuración de los modelos locales y los materiales necesarios para reproducir el despliegue. Se menciona aquí de forma explícita porque una parte importante del valor del trabajo reside precisamente en la combinación de componentes reutilizados con código propio integrador.

1.4 Objetivos del proyecto

El objetivo principal de este trabajo es desarrollar un asistente conversacional corporativo sustentado en una arquitectura RAG *on-premise*, capaz de consultar documentación cambiante almacenada en la nube o en otras fuentes heterogéneas sin sacar los datos del entorno controlado de la organización. Aunque la idea general es poder trabajar con repositorios documentales de distinto tipo, la implementación y las pruebas de este TFG se han realizado principalmente sobre SharePoint como caso real de integración.

La motivación operativa es clara: si los problemas detectados son dispersión documental, formatos heterogéneos, riesgo de *Shadow AI* y falta de trazabilidad, la respuesta técnica debe ser un sistema que recupere información de forma centralizada, la explique con lenguaje natural y deje claro en qué documentos se apoya.

Para alcanzar este objetivo general, se han definido los siguientes objetivos específicos:

1. *Diseñar una arquitectura modular reutilizando bloques ya disponibles y añadiendo desarrollo propio donde aporta valor*: combinar modelos de lenguaje locales, embeddings, una base de datos vectorial y una interfaz conversacional, añadiendo un backend en Python que orqueste todo el sistema.

[1] https://github.com/acidxlemons/TFG-JARVIS

2. *Permitir ingesta multicanal y actualización de conocimiento*: incorporar carga y actualización documental desde fuentes heterogéneas, con especial foco en SharePoint, incluyendo OCR para documentos escaneados y extracción web cuando sea necesario.

3. *Ofrecer respuestas fundamentadas y trazables*: implementar un flujo RAG que priorice la recuperación documental frente a la generación libre y muestre al usuario sobre qué fuentes se construye cada respuesta.

4. *Añadir capacidades complementarias sin entrenar modelos desde cero*: integrar búsqueda web controlada, scraping, análisis de imágenes y consulta normativa del BOE reutilizando herramientas ya disponibles y conectándolas mediante lógica propia.

5. *Explorar MCP como línea de integración futura*: desarrollar un servidor compatible con el enfoque de Model Context Protocol (MCP) [6] para el caso del BOE, dejando preparada una vía de evolución aunque la interfaz web del proyecto siga usando integración por API en la implementación actual.

6. *Desplegar el sistema de forma reproducible y observable*: contenerizar el ecosistema con Docker y habilitar métricas y paneles de monitorización que permitan operar y analizar el sistema.

1.5 Alcance y limitaciones

1.5.1 Alcance

El proyecto cubre el ciclo completo de construcción de un sistema RAG empresarial, desde el diseño de la arquitectura hasta su despliegue reproducible. El alcance real del trabajo se resume en los siguientes puntos:

- Selección y despliegue de modelos existentes de lenguaje, visión y embeddings; no se entrena un modelo fundacional desde cero.

- Diseño e implementación de una arquitectura basada en microservicios contenerizados.

- Desarrollo de un backend propio en Python para orquestación, recuperación documental, control de acceso y conexión con herramientas externas.

- Integración de un pipeline de ingesta documental con soporte para PDFs, HTML, imágenes y documentación alojada en SharePoint.

- Construcción de un motor de búsqueda semántica basado en embeddings y almacenamiento vectorial.
- Reutilización de OpenWebUI como interfaz conversacional e integración de su autenticación con el proveedor corporativo de identidad mediante OIDC/Azure AD.
- Extensión de OpenWebUI mediante un pipeline propio que añade encaminamiento, memoria de sesión y control de acceso documental.
- Integración funcional con fuentes externas como SharePoint, BOE e Internet.
- Desarrollo de un servidor MCP para normativa del BOE como prueba de concepto técnica orientada a futuras integraciones.
- Incorporación de un stack de monitorización y observabilidad para analizar uso y rendimiento.

En otras palabras, el alcance combina dos tipos de trabajo. Por un lado, se reutilizan componentes ya maduros, como OpenWebUI, Ollama, Qdrant o PaddleOCR. Por otro, se desarrollan específicamente para el TFG las piezas que faltaban para convertirlos en un sistema coherente: el backend RAG, el pipeline JARVIS, el indexador, la integración BOE y la configuración operativa del despliegue.

En relación con MCP, conviene precisar el alcance para evitar ambigüedades: en este TFG no se desarrolla un cliente MCP completo dentro de la interfaz web, sino un servidor MCP propio validado en entorno compatible. La funcionalidad visible por el usuario final se presta actualmente mediante llamadas a API, mientras que MCP queda como capacidad implementada y preparada para evolucionar.

1.5.2 Limitaciones

Se identifican las siguientes limitaciones reconocidas:

- *Requisitos hardware*: el sistema requiere una GPU NVIDIA con suficiente VRAM para mantener un rendimiento aceptable en tareas de OCR e inferencia local.
- *Predominio de SharePoint en la validación*: aunque la arquitectura está pensada para documentos cambiantes en la nube y de distinta naturaleza, la validación experimental se ha centrado sobre todo en SharePoint, que es la integración más desarrollada del proyecto.

- *BOE con doble nivel de integración*: el sistema permite consultas normativas desde la interfaz mediante API, pero la integración MCP nativa en la web no se incorpora todavía por limitaciones del cliente utilizado.
- *MCP validado fuera de la interfaz principal*: el servidor MCP desarrollado funciona en entorno de terminal con clientes compatibles, pero su adopción dentro de la interfaz web queda como trabajo futuro.
- *Sesgo en el ajuste fino*: el *fine-tuning* LoRA se realiza con datos sintéticos generados a partir del propio sistema base, lo que puede introducir sesgos y limitar la generalización.
- *Evaluación acotada*: la evaluación se realiza principalmente de forma funcional y cualitativa; benchmarks académicos más formales quedan fuera del alcance temporal del proyecto.
- *Obsolescencia rápida del contexto tecnológico*: el ecosistema de modelos, herramientas y estándares evoluciona a gran velocidad, de modo que algunas decisiones tecnológicas razonables durante el desarrollo pueden quedar superadas en pocos meses.

1.6 Contribuciones principales

Las contribuciones técnicas concretas de este trabajo son:

1. Una arquitectura reproducible basada en microservicios, definida declarativamente en Docker Compose, con servicios orientados a ingesta, almacenamiento, recuperación, generación y monitorización.
2. Un agente encaminador, JARVIS, capaz de seleccionar distintos modos de operación en función de la intención detectada y de guiar cada consulta hacia el subsistema adecuado.
3. Un pipeline de ingesta multicanal que unifica SharePoint, scraping web, OCR y carga manual en una única base de conocimiento vectorial.
4. Una integración funcional con el BOE dentro del sistema, permitiendo búsquedas y consultas normativas desde la experiencia conversacional.
5. El desarrollo y validación de un servidor MCP para consulta normativa del BOE, operativo fuera de la interfaz principal y planteado como base de evolución futura.

6. Un flujo de adaptación de modelos base mediante LoRA orientado al dominio del proyecto, evaluado como complemento y no como núcleo de la solución.

7. La publicación del sistema como proyecto de código abierto en GitHub[2], con documentación para despliegue reproducible.

1.7 Estructura de la memoria

El resto de este documento se estructura de la siguiente manera:

En el **Capítulo 2** se presentan los fundamentos que permiten entender la solución propuesta: qué son los LLMs, por qué un LLM por sí solo no basta en este problema y qué papel desempeñan RAG, los embeddings, las bases vectoriales, la cuantización, LoRA y MCP.

El **Capítulo 3** aterriza las tecnologías concretas utilizadas en el proyecto, explicando para qué sirve cada una, por qué se ha elegido y cuál es su papel dentro del sistema completo.

El **Capítulo 4** analiza sistemas existentes similares al desarrollado, tanto comerciales como de código abierto, para situar la propuesta dentro del ecosistema actual.

El **Capítulo 5** ofrece una descripción funcional completa del sistema desde la perspectiva del usuario: interfaz conversacional, modos de operación, escenarios de uso y panel de administración.

El **Capítulo 6** constituye el núcleo técnico del proyecto. Describe la arquitectura, la separación entre desarrollo propio y herramientas externas, el mecanismo Pipeline de OpenWebUI, el agente JARVIS, el motor RAG y las integraciones principales.

El **Capítulo 7** describe la metodología seguida, las iteraciones de desarrollo, el entorno hardware, el stack tecnológico completo, la estimación de presupuesto y la estrategia de documentación.

El **Capítulo 8** presenta los resultados obtenidos: métricas de rendimiento, evaluación funcional y análisis del comportamiento del sistema.

Por último, el **Capítulo 9** recoge las conclusiones, el grado de consecución de los objetivos y las líneas futuras de trabajo.

[2] https://github.com/acidxlemons/TFG-JARVIS

Capítulo 2

Fundamentos de la solución propuesta

Este capítulo no pretende enumerar todas las tecnologías del sector, sino introducir únicamente los conceptos necesarios para entender la solución desarrollada en este TFG. La idea es guiar al lector desde el problema hasta la arquitectura elegida: primero se explica qué puede aportar un LLM y por qué no basta por sí solo; después se presenta RAG como mecanismo para responder con apoyo documental; y finalmente se describen los componentes que permiten ejecutar todo ello de forma local, controlada y útil en un contexto corporativo.

2.1 Grandes Modelos de Lenguaje (LLMs)

Los grandes modelos de lenguaje (LLMs) son modelos neuronales entrenados para predecir el siguiente *token* en una secuencia. El salto de rendimiento moderno llega con la arquitectura *Transformer* [7], donde el mecanismo de atención permite capturar relaciones de contexto con mucha más eficacia que arquitecturas previas.

Desde el punto de vista del usuario, un LLM permite dialogar en lenguaje natural, resumir contenido, reformular instrucciones o generar texto coherente. Sin embargo, conviene aclarar un punto importante para este trabajo: un LLM no es una base de datos ni un buscador documental. Si se le pregunta por información corporativa que no forma parte de su entrenamiento o que ha cambiado recientemente, tenderá a responder con estimaciones plausibles, no necesariamente con hechos verificables.

2.1.1 Propietarios vs. pesos abiertos

En la práctica, existen dos formas habituales de utilizar LLMs:

- *Modelos propietarios vía API*: ofrecen un producto muy maduro y alto rendimiento, pero implican dependencia de un proveedor externo y procesamiento fuera del perímetro corporativo.
- *Modelos de pesos abiertos (open-weights)*: permiten ejecución local u *on-premise*, lo que habilita mayor control sobre datos, configuración y despliegue.

En este proyecto se prioriza el segundo enfoque porque el requisito principal es operar en local. También conviene diferenciar entre *pesos abiertos* y *código abierto*: que un modelo pueda descargarse y ejecutarse localmente no implica necesariamente una licencia plenamente abierta [8], [9], [10].

El sistema desarrollado reutiliza modelos ya existentes, principalmente de la familia Qwen 2.5 y, en fases previas del proyecto, también Llama 3.1, por ser viables en despliegue local y contar con un ecosistema maduro [11], [12]. Esta decisión refuerza la idea central del proyecto: el valor no está en crear un nuevo modelo fundacional, sino en integrarlo correctamente con el conocimiento de la organización.

2.2 Por qué un LLM por sí solo no resuelve este problema

El problema abordado en este TFG no consiste en mantener una conversación general, sino en responder sobre documentación interna, normativa y contenidos que cambian con el tiempo. Un LLM aislado presenta varias limitaciones en este escenario:

- No conoce por defecto los documentos privados de la organización.
- No accede en tiempo real a repositorios documentales salvo que se le conecte explícitamente a ellos.
- Puede mezclar conocimiento general con suposiciones plausibles.
- No ofrece trazabilidad automática hacia los documentos utilizados.

Por ello, aunque los LLMs son la pieza que permite la interacción en lenguaje natural, no constituyen por sí mismos la solución completa. Hace falta una arquitectura adicional que recupere información, filtre fuentes, construya contexto y limite la generación a aquello que realmente se ha encontrado.

2.3 Generación Aumentada mediante Recuperación (RAG)

La arquitectura que responde a ese problema es RAG (*Retrieval-Augmented Generation*) [1]. Su idea básica es sencilla: antes de generar una respuesta, el sistema busca fragmentos relevantes de documentos y se los proporciona al modelo como contexto. De esta forma, el LLM deja de apoyarse solo en su memoria estadística y pasa a responder utilizando evidencia concreta.

Un pipeline RAG típico tiene dos fases:

1. *Recuperación*: la pregunta del usuario se transforma en una representación numérica y se buscan fragmentos relevantes en una base documental indexada.

2. *Generación*: el LLM recibe la pregunta junto con ese contexto recuperado y redacta una respuesta basada en él.

Para un lector no técnico, puede entenderse como una combinación entre un buscador y un asistente conversacional. El buscador encuentra la información; el asistente la explica. Esa combinación es la razón por la que RAG encaja mejor en este proyecto que un chatbot convencional.

2.3.1 Mejoras prácticas: búsqueda híbrida y memoria acotada

En un sistema real, RAG no suele quedarse en una búsqueda semántica simple:

- *Búsqueda híbrida*: combina una recuperación basada en significado, obtenida al comparar embeddings, con otra basada en coincidencias exactas de palabras, habitualmente implementada con métodos como BM25. Esto evita perder siglas, identificadores, nombres de procedimientos o artículos legales [13], [14], [15].

- *Reordenación y filtrado*: se seleccionan los fragmentos más útiles antes de enviarlos al modelo.

- *Memoria conversacional acotada*: se conserva parte del historial cuando aporta contexto, pero evitando que el *prompt* crezca sin control.

La solución de este TFG utiliza precisamente esa variante práctica de RAG, porque es la que mejor se adapta a consultas corporativas reales.

2.4 Modelos de Embeddings

Los embeddings convierten texto en vectores numéricos donde la cercanía entre vectores representa similitud semántica. De forma intuitiva, puede pensarse que cada texto se transforma en una huella numérica compacta: si dos huellas quedan cerca en ese espacio, sus significados tienden a ser parecidos. Son el mecanismo que permite pasar de una pregunta escrita por una persona a una búsqueda de contenido relacionada por significado y no solo por palabras exactas.

Una medida habitual de similitud es la similitud del coseno:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}|| \cdot ||\mathbf{b}||} \tag{2.1}$$

En este proyecto se emplean modelos de SentenceTransformers [16], [17], ya existentes y desplegables en local, porque permiten generar embeddings sin depender de servicios externos. Esta decisión es importante por dos razones: preserva la privacidad y desacopla la búsqueda semántica del proveedor del LLM generativo.

También se explora la adaptación del modelo de embeddings al dominio del proyecto. Esto es relevante porque, en un entorno corporativo, términos internos, acrónimos y nomenclatura especializada pueden no estar bien representados por un modelo genérico.

2.5 Bases de datos vectoriales

Una vez calculados los embeddings, hace falta un sistema capaz de almacenarlos y consultarlos con rapidez. Ese papel lo cumplen las bases de datos vectoriales. En lugar de buscar solo texto, permiten buscar vectores parecidos al vector de la consulta.

En colecciones grandes se usan índices aproximados para acelerar la búsqueda. Uno de los enfoques más extendidos es HNSW [18], un grafo navegable jerárquico diseñado para encontrar vecinos cercanos de forma eficiente.

Tabla 2.1: Comparativa de bases de datos vectoriales evaluadas para el proyecto.

Base de datos	**Licencia**	**On-Premise**	**Filtrado**	**Híbrida**
Qdrant	Apache 2.0	Sí	Sí	Sí
Weaviate	BSD-3	Sí	Sí	Sí
Milvus	Apache 2.0	Sí	Sí	Sí
Chroma	Apache 2.0	Sí	Limitado	Limitado
Pinecone	Propietaria	No (SaaS)	Sí	Parcial

Se selecciona Qdrant por ser *on-premise*, por su filtrado por metadatos y por su buen soporte para búsqueda híbrida [15], [19]. Para este TFG no es solo una decisión de rendimiento, sino también de control: permite separar colecciones, añadir metadatos y aplicar restricciones de acceso documentales.

2.6 Cuantización y ejecución local

Ejecutar LLMs localmente exige gestionar bien los recursos de hardware, especialmente la VRAM de la GPU. La cuantización reduce la precisión numérica de los pesos para disminuir consumo de memoria y hacer viable la inferencia en hardware más modesto.

En el ecosistema local, el formato GGUF se utiliza ampliamente para empaquetar modelos cuantizados junto con sus metadatos [20], [21]. Herramientas como Ollama permiten servir estos modelos por API y simplifican mucho el despliegue [22].

Este concepto es importante para entender una de las decisiones prácticas del proyecto: no basta con elegir un buen modelo, también hay que elegir una variante que realmente pueda ejecutarse en el hardware disponible sin bloquear el resto del sistema.

2.7 Ajuste fino eficiente con LoRA

El ajuste fino (*fine-tuning*) adapta un modelo generalista a un dominio concreto. Sin embargo, reentrenar por completo un LLM moderno está fuera del alcance normal de un TFG y tampoco era el objetivo principal de este trabajo.

La técnica LoRA (*Low-Rank Adaptation*) reduce drásticamente ese coste reentrenando solo una pequeña parte de la red neuronal del modelo, en lugar de modificar todos sus pesos [23]. Concretamente, inserta capas adicionales de tamaño muy reducido —matrices de bajo rango— en las capas de atención del transformador y reentrena únicamente esas capas añadidas, dejando los pesos originales congelados. En la práctica, puede entenderse como ampliar la red neuronal con nuevas conexiones especializadas sin tener que reentrenar toda la arquitectura de partida.

Formalmente, el cambio en el peso de una capa se expresa como:

$$W' = W + \Delta W = W + BA \tag{2.2}$$

donde A y B son matrices de rango reducido con $r \ll \text{mín}(d, k)$, lo que hace que el número de parámetros entrenables pase de millones a miles.

En este proyecto, LoRA se estudia como una vía de adaptación incremental sobre modelos ya existentes, no como el eje principal de la solución. La experiencia obtenida

muestra que, para consultas RAG puras, suele tener más impacto un buen contexto recuperado y un buen modelo de embeddings que modificar directamente los pesos del LLM generativo.

2.8 Model Context Protocol (MCP)

MCP es un estándar abierto propuesto por Anthropic en 2024 para integrar herramientas externas con agentes basados en LLM mediante una interfaz cliente-servidor tipada [24]. Un servidor MCP expone herramientas y un cliente compatible puede descubrirlas e invocarlas de forma estructurada.

Para este TFG, MCP es relevante por dos motivos. Primero, porque representa una forma moderna de conectar modelos con herramientas externas. Segundo, porque permite aclarar el alcance del trabajo: el proyecto no implementa una plataforma MCP completa desde cero, sino un *servidor MCP propio* orientado a consulta normativa del BOE, desarrollado como prueba de integración real.

Ese servidor se ha validado en ejecución por terminal con clientes compatibles. Además, OpenWebUI ya ofrece soporte nativo para servidores MCP en versiones recientes [25]. Aun así, en este TFG la funcionalidad visible dentro de la interfaz web sigue resolviéndose principalmente mediante API tradicional y pipeline propio, porque esa fue la vía integrada y validada durante el desarrollo para mantener mayor control sobre autenticación, encaminamiento y operación del sistema. Por tanto, MCP aparece en este trabajo como tecnología implementada y validada, pero todavía no como flujo principal de uso para el usuario final.

Capítulo 3

Tecnologas utilizadas y criterio de eleccin

Este captulo aterriza la solucin en tecnologas concretas. Su objetivo principal es describir qu es cada tecnologa y qu papel desempea dentro del sistema completo. Cuando resulta til, se menciona tambin por qu encaja en el proyecto, pero el detalle de cmo se ha combinado con otras piezas se deja para el Captulo 6. Tambin resulta importante distinguir entre herramientas preexistentes reutilizadas en el proyecto y componentes desarrollados especficamente en este TFG.

3.1 Python como lenguaje de integracin principal

El lenguaje vertebrador del proyecto es Python. La razn principal no es solo su popularidad, sino su capacidad para actuar como lenguaje de integracin entre bibliotecas de IA, servicios web, automatizacin y procesamiento documental. En este TFG, Python permite conectar en un mismo ecosistema el backend, el indexador, el pipeline de OpenWebUI, la integracin con SharePoint, el scraping, el OCR y el servidor MCP.

Conviene separar aqu dos planos. Por un lado, hay paquetes que se usan directamente en el cdigo desarrollado en el TFG, como FastAPI, qdrant-client, sentence-transformers o msal. Por otro, hay dependencias que vienen dadas por herramientas ya reutilizadas en el stack, como parte del ecosistema de OpenWebUI, Docker o Grafana. Esta distincin es importante porque el proyecto no consiste en construir esas plataformas de base, sino en apoyarse en ellas y extenderlas donde era necesario.

Adems, buena parte de las herramientas reutilizadas en el proyecto exponen su mejor soporte precisamente en este lenguaje. Entre los paquetes y bibliotecas ms importantes utilizados se encuentran:

- **FastAPI** y **Uvicorn** para los servicios web y la API REST.
- **Pydantic** para validacin de datos y configuracin tipada.
- **sentence-transformers** para generacin de embeddings.
- **qdrant-client** para interaccin con la base de datos vectorial.
- **msal** y clientes asociados para autenticacin y acceso a Microsoft Graph.
- **playwright** para automatizacin de navegador.
- **trafilatura**, **readability** y **BeautifulSoup** para extraccin de contenido web.
- **PaddleOCR** para OCR de documentos escaneados.
- **duckduckgo_search** para bsqueda web controlada.
- **fastmcp** para el servidor MCP del BOE.
- **prometheus-client** para exponer mtricas del backend.

Esta eleccin tambin ayuda a explicar la naturaleza del trabajo realizado: el proyecto no consiste en inventar estas bibliotecas, sino en utilizarlas de forma coordinada y construir sobre ellas la lgica que faltaba para el caso de uso corporativo.

3.2 OpenWebUI: interfaz conversacional reutilizada

OpenWebUI[1] es la interfaz conversacional utilizada como punto de entrada del sistema [26]. Es una herramienta externa, ya existente, que he reutilizado para no invertir el esfuerzo del TFG en construir una interfaz de chat desde cero. Esta decisin permite concentrar el trabajo en la parte realmente diferencial del proyecto: la integracin con las fuentes documentales y la lgica del asistente.

3.2.1 Arquitectura interna de OpenWebUI

OpenWebUI se estructura como una aplicacin web con tres bloques principales:

- *Frontend*: interfaz tipo chat accesible desde navegador, con historial de conversaciones, carga de adjuntos y streaming de respuestas.
- *Backend*: lgica de autenticacin, persistencia de chats y gestin de modelos.
- *Base de datos*: almacenamiento de usuarios, conversaciones y metadatos.

Para el lector, lo importante es entender su papel dentro del proyecto: OpenWebUI aporta la experiencia de usuario, pero no constituye la inteligencia principal del sistema. Esa inteligencia se aade mediante el pipeline desarrollado en este TFG.

En consecuencia, cuando en esta memoria se habla de “integrar autenticacin corporativa con la interfaz conversacional”, no se est describiendo la creacin de una interfaz nueva, sino la reutilizacin de OpenWebUI junto con su conexin al proveedor corporativo de identidad y su ampliacin mediante un pipeline propio.

[1] https://openwebui.com/

3.2.2 Por qu OpenWebUI

La eleccin de OpenWebUI se justifica porque ya ofrece varias capacidades deseables en un entorno corporativo: interfaz madura, persistencia de conversaciones, compatibilidad con modelos locales y soporte de autenticacin. Frente a hacer una interfaz propia, reduce tiempo de desarrollo y riesgo tcnico.

Tabla 3.1: Comparativa de interfaces de usuario para LLMs locales.

Criterio	**OpenWebUI**	**Gradio**	**Streamlit**	**ChatUI**
UI tipo ChatGPT	✓	Parcial	Parcial	✓
Compatibilidad con Ollama	Nativa	Manual	Manual	Nativa
Sistema de Pipelines/Plugins	✓	×	×	×
Gestin de usuarios	✓	×	×	✓
Persistencia de chats	✓	×	×	✓
SSO / OIDC	✓	×	×	×
Cdigo abierto	✓	✓	✓	✓
Carga de archivos adjuntos	✓	✓	✓	Parcial

El factor decisivo es el sistema de Pipelines, porque permite insertar lgica personalizada sin modificar el ncleo de la herramienta. Gracias a ello, la interfaz sigue siendo una pieza reutilizada, pero el comportamiento final del asistente pasa a estar controlado por desarrollo propio.

3.2.3 Sistema de Pipelines de OpenWebUI

El sistema de Pipelines es el mecanismo que convierte una interfaz genrica en un asistente especializado. Funciona como una capa intermedia entre el usuario y el modelo: recibe el mensaje, decide qu hacer con l y devuelve la respuesta final.

Desde el punto de vista del proyecto, esta capacidad es clave porque permite:

- interceptar cada consulta antes de enviarla al modelo,
- aplicar lgica de encaminamiento,
- conectar con el backend propio,
- y conservar la interfaz original sin reescribirla.

Este es uno de los mejores ejemplos de la filosofa del TFG: reutilizar una herramienta existente cuando ya resuelve bien una capa del problema y centrar el desarrollo propio en la parte que aporta valor diferencial.

3.3 Ollama: ejecucin local de modelos

Ollama[2] es la herramienta externa utilizada para ejecutar modelos de lenguaje localmente. Su funcin es cargar modelos, servirlos por API y gestionar su uso en el hardware disponible [27].

3.3.1 Caractersticas tcnicas

Se ha elegido Ollama por varias razones prcticas:

- *Facilita el despliegue de modelos locales*: evita construir una capa de inferencia propia desde cero.
- *Gestiona la VRAM de manera operativa*: carga y descarga modelos segn sea necesario.
- *Expone una API sencilla*: esto simplifica mucho su integracin con el resto del sistema.
- *Encaja bien en contenedores*: puede desplegarse como un servicio ms dentro de Docker Compose.

3.3.2 Modelos desplegados

En el sistema se emplean varias familias de modelos ya existentes, cada una con una funcin concreta:

Tabla 3.2: Modelos de IA desplegados en Ollama y su funcin en el sistema.

Modelo	**Funcin**	**Cuantizacin**	**VRAM**
rag-qwen-ft:latest	Modelo principal del alias JARVIS para chat y RAG	LoRA FT	∼8 GB
qwen2.5:32b-instruct-q4_K_M	Fallback de texto y comparacin base	Q4_K_M	∼19 GB
qwen2.5-coder:32b	Modelo auxiliar instalado para tareas de cdigo	—	∼19 GB
qwen2.5vl:7b	Anlisis de imgenes y OCR multimodal	—	∼6 GB

La tabla refuerza una idea importante: los modelos no se han creado en este TFG, pero s se han seleccionado, desplegado, probado y asignado a tareas concretas dentro de una arquitectura coherente. En el despliegue actual del proyecto, la familia Qwen 2.5 concentra las funciones principales de texto, visin y adaptacin al dominio.

[2] https://ollama.com/

3.4 Qdrant: base de datos vectorial

Qdrant[3] es la base de datos vectorial empleada como almacn principal del conocimiento documental del sistema [28]. Es una herramienta externa, pero su estructura, colecciones y criterios de uso se definen dentro del proyecto.

3.4.1 Caractersticas utilizadas

- *ndice HNSW*: permite bsquedas vectoriales rpidas sobre grandes volmenes de fragmentos.
- *Bsqueda hbrida*: combina similitud semntica y coincidencia lxica exacta.
- *Filtrado por metadatos*: habilita segmentacin por fuente, documento o departamento.
- *Multi-coleccin*: permite separar documentos por contexto de uso.

Para el proyecto, esto significa que Qdrant no es solo un repositorio tcnico, sino el componente que hace posible que el sistema encuentre evidencia relevante antes de responder.

3.5 FastAPI: backend del sistema

FastAPI[4] es el framework seleccionado para construir los servicios backend en Python [29]. A diferencia de OpenWebUI, Ollama o Qdrant, que son herramientas reutilizadas, aqu s aparece una parte importante de desarrollo propio: los servicios implementados con FastAPI contienen gran parte de la lgica central del sistema.

Se ha elegido por tres motivos principales:

- **Facilidad para construir APIs tipadas y mantenibles**.
- *Soporte asncrono*: til cuando el sistema depende de mltiples llamadas a servicios externos o internos.
- *Integracin natural con el ecosistema Python*: permite usar en el mismo entorno paquetes de IA, scraping, OCR y autenticacin.

En este TFG, FastAPI sustenta el backend RAG, el indexador de documentos y parte de las integraciones auxiliares.

[3] https://qdrant.tech/
[4] https://fastapi.tiangolo.com/

3.6 LiteLLM: proxy unificado de modelos

LiteLLM[5] es una herramienta externa que se utiliza como capa intermedia entre los servicios propios y los modelos servidos por Ollama [30]. Su utilidad est en unificar llamadas, aplicar alias de modelos, gestionar cach y simplificar el encaminamiento tcnico.

Desde la perspectiva del lector no tcnico, su papel puede resumirse as: en lugar de que cada parte del sistema tenga que conocer todos los detalles de cada modelo, LiteLLM ofrece un punto nico y homogneo de acceso.

3.7 Docker y orquestacin de contenedores

Docker[6] y Docker Compose son la base del despliegue reproducible [31]. Se utilizan para empaquetar todos los componentes y ponerlos en marcha como un conjunto coordinado.

3.7.1 Ventajas de la contenerizacin

- *Reproducibilidad*: el mismo sistema puede desplegarse de forma controlada en distintos entornos.
- *Aislamiento*: cada servicio mantiene sus dependencias separadas.
- *Seguridad perimetral*: solo ciertos servicios se exponen al exterior.
- *Acceso a GPU cuando es necesario*: se reserva aceleracin solo para los servicios que la necesitan.

En el contexto del TFG, esto tambin es una parte de la contribucin: no solo se han elegido tecnologas, sino que se han dejado integradas dentro de una arquitectura desplegable de forma repetible.

3.8 Herramientas de procesamiento documental y web

Esta seccin rene herramientas ya existentes que no han sido desarrolladas en el proyecto, pero que s se han integrado y combinado para cubrir necesidades concretas del sistema.

[5] https://docs.litellm.ai/
[6] https://www.docker.com/

3.8.1 PaddleOCR: reconocimiento ptico de caracteres

PaddleOCR[7] es un motor OCR externo utilizado para extraer texto de PDFs escaneados [32]. No se ha implementado un OCR propio; el trabajo realizado aqu consiste en decidir cundo usarlo, cmo integrarlo en la ingesta y cmo hacerlo convivir con el resto de componentes del sistema.

3.8.2 Playwright: automatizacin de navegadores

Playwright[8] es una herramienta externa de automatizacin de navegador [33]. Se utiliza porque muchas pginas web modernas dependen de JavaScript y no pueden procesarse correctamente con una simple peticin HTTP. Su papel dentro del proyecto es permitir obtener el contenido real que ve el usuario en pginas dinmicas.

3.8.3 Trafilatura: extraccin de contenido web

Trafilatura [34] es una biblioteca externa especializada en extraer el contenido principal de una pgina web, eliminando mens, barras laterales y ruido. En este TFG no se ha desarrollado un extractor propio de texto web desde cero; se ha integrado Trafilatura como primera opcin dentro de una cadena de extraccin con *fallbacks*, porque resuelve bien el problema de aislar el contenido relevante.

3.8.4 SentenceTransformers: generacin de embeddings

SentenceTransformers [16], [17] es una biblioteca externa utilizada para generar embeddings. Se emplea el modelo `paraphrase-multilingual-MiniLM-L12-v2`, elegido por su equilibrio entre tamao, soporte multilinge y facilidad de despliegue. Aqu tambin es importante la distincin metodolgica: el modelo y la librera ya existan; lo que aporta el proyecto es su integracin en el pipeline RAG y su ajuste al caso de uso documental.

3.9 Stack de monitorizacin

Para operar un sistema de este tipo no basta con que funcione: tambin hay que entender cmo se comporta. Por ello se incorpora un stack de observabilidad formado por herramientas ya existentes, configuradas y conectadas dentro del proyecto.

[7] https://github.com/PaddlePaddle/PaddleOCR
[8] https://playwright.dev/

3.9.1 Prometheus

Prometheus[9] recoge mtricas temporales del backend y de otros servicios [35]. Su utilidad en este TFG es permitir medir latencia, tasa de errores y patrones de uso en lugar de basar las decisiones de optimizacin en impresiones subjetivas.

3.9.2 Grafana

Grafana[10] visualiza las mtricas recogidas por Prometheus mediante paneles comprensibles [36]. En el proyecto se ha utilizado para construir paneles especficos de backend, GPU y bases de datos. Esto no debe entenderse como un desarrollo de una herramienta nueva, sino como el diseo de la capa de observacin necesaria para operar el sistema con criterio.

3.9.3 NVIDIA DCGM Exporter

NVIDIA DCGM Exporter[11] expone mtricas de la GPU en formato compatible con Prometheus [37]. Su presencia se justifica porque en un sistema que ejecuta modelos y OCR en local, la GPU es un recurso crtico y debe monitorizarse con el mismo nivel de detalle que el backend.

3.10 Otras tecnologas del ecosistema

- *PostgreSQL*: almacena usuarios, conversaciones y metadatos de OpenWebUI.
- *Redis*: se emplea como cach de respuestas a travs de LiteLLM.
- *MinIO*: conserva documentos binarios como respaldo y fuente de reindexacin.
- *NGINX*: acta como proxy inverso y punto nico de entrada.
- *MSAL*: permite autenticacin con Azure AD y acceso autenticado a SharePoint mediante Microsoft Graph.

[9] https://prometheus.io/
[10] https://grafana.com/
[11] https://github.com/NVIDIA/dcgm-exporter

3.11 Diagrama de flujo de tecnologas usadas

La Figura 3.1 resume cmo se conectan las tecnologas principales del sistema. No representa nicamente el recorrido de una consulta, sino tambin los componentes de soporte que permiten que el sistema funcione en un entorno real: autenticacin, almacenamiento, cach y monitorizacin. Se incluye aqu porque ayuda a que un lector no especializado entienda de un vistazo qu piezas existen y cmo se relacionan.

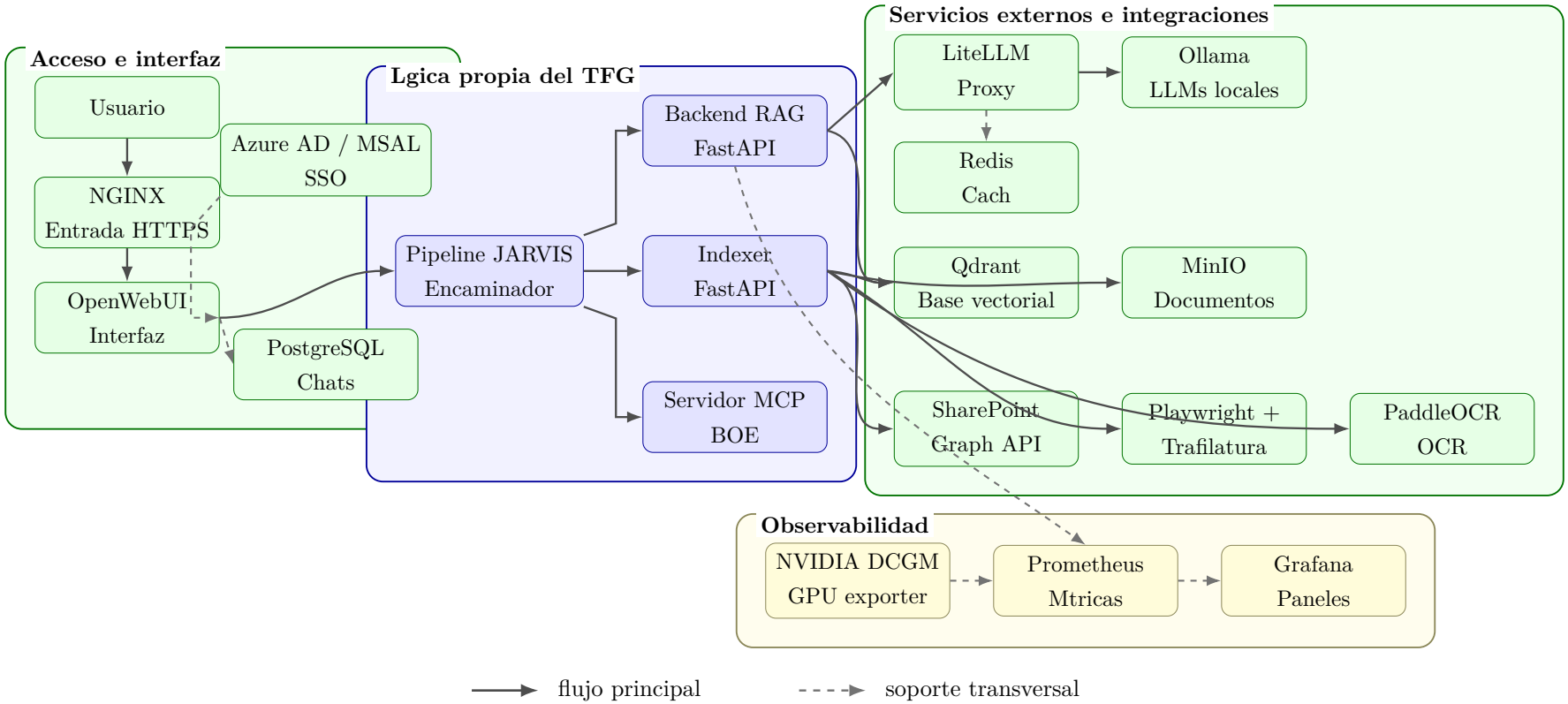

Figura 3.1: Diagrama general de las tecnologas utilizadas en el sistema. La disposicin apaisada separa el flujo de consulta, la lgica propia del TFG, las integraciones externas y la capa de observabilidad en bloques legibles.

Capítulo 4

Sistemas relacionados y posicionamiento de la propuesta

Este capítulo sitúa el sistema desarrollado dentro del ecosistema actual. El objetivo no es comparar por comparar, sino mostrar qué soluciones existían antes de comenzar el proyecto, cuáles han aparecido o ganado relevancia durante los últimos meses y qué hueco intenta cubrir la propuesta de este TFG. Esta parte es especialmente importante porque el ámbito de los asistentes con LLM evoluciona muy rápido: herramientas que hace pocos meses no estaban maduras hoy ya compiten en funcionalidades concretas.

4.1 Asistentes comerciales basados en LLM

4.1.1 ChatGPT y GPT Enterprise (OpenAI)

ChatGPT es el asistente conversacional más conocido del mercado. Su versión empresarial añade controles administrativos, garantías comerciales y mejores condiciones de tratamiento de datos [38]. Aun así, sigue siendo un servicio en la nube: la organización no controla la infraestructura de inferencia ni puede modificar en profundidad la lógica del sistema.

Para este TFG, el principal límite de esta familia de soluciones no es su calidad técnica, sino la pérdida de soberanía operativa: los documentos y consultas no se procesan dentro de una infraestructura propia.

4.1.2 Microsoft Copilot para Microsoft 365

Microsoft Copilot destaca por su integración con SharePoint, Outlook, Teams y el resto del ecosistema Microsoft 365 [39]. Para organizaciones ya asentadas en ese entorno, su propuesta es muy atractiva porque reduce la fricción de adopción.

Sin embargo, esa misma integración va asociada a varias restricciones relevantes para el enfoque de este trabajo:

- procesamiento en nube, no en infraestructura propia;
- coste por usuario recurrente;
- escasa capacidad para modificar la lógica interna del asistente;
- dependencia fuerte del ecosistema Microsoft.

4.1.3 Google Gemini y Vertex AI

Google ofrece Gemini como producto de productividad y Vertex AI como plataforma de desarrollo e integración [40], [41]. Su fortaleza está en la amplitud del ecosistema cloud y en la disponibilidad de servicios gestionados.

Desde la perspectiva de este proyecto, la limitación vuelve a ser similar: son plataformas muy capaces, pero diseñadas principalmente para operar en la nube del proveedor, no para una arquitectura enteramente *on-premise*.

4.1.4 Amazon Bedrock

Amazon Bedrock proporciona acceso a varios modelos mediante una API unificada y ofrece piezas para agentes y RAG [42]. Es una solución flexible dentro del ecosistema AWS, pero sigue respondiendo al paradigma de servicio gestionado en la nube.

4.2 Herramientas abiertas orientadas a RAG y asistentes privados

4.2.1 PrivateGPT

PrivateGPT [43] representa una de las aproximaciones más conocidas a la idea de chat privado con documentos propios. Comparte con este TFG la filosofía de evitar servicios externos, pero está más orientado a un uso individual o de laboratorio que a un despliegue corporativo completo.

4.2.2 AnythingLLM

AnythingLLM [44] combina interfaz, RAG y gestión documental en una sola herramienta. Se acerca bastante a una solución práctica lista para usar, pero su comportamiento es más uniforme y menos controlable que el del sistema de este proyecto, donde existe un pipeline propio de encaminamiento e integración con fuentes específicas.

4.2.3 Danswer / Onyx

Danswer, renombrado posteriormente como Onyx, es una de las referencias más cercanas a un buscador corporativo con LLM y múltiples conectores [45]. Su relevancia es alta porque demuestra que la necesidad que aborda este TFG no es académica, sino real y compartida por muchas organizaciones.

La diferencia principal es de enfoque: Onyx persigue un producto generalista para empresa; este TFG construye una arquitectura más controlada y personalizada alrededor de un caso concreto, con ejecución local, pipeline propio y mayor énfasis en componentes desplegables por separado.

4.2.4 OpenClaw y la aceleración reciente del ecosistema

Durante los últimos meses han aparecido o se han popularizado nuevas herramientas abiertas orientadas a agentes, automatización y uso local de modelos. Un ejemplo representativo es OpenClaw [46], que se centra en conectar agentes con múltiples canales de mensajería y refleja una tendencia clara del sector: pasar de simples chats con documentos a sistemas más capaces de orquestar herramientas, operar en varios canales y ejecutar tareas más complejas.

Su mención es importante por dos razones. La primera es temporal: muestra que el espacio tecnológico del proyecto se está moviendo muy deprisa y que algunas referencias actuales no estaban igual de maduras cuando se definió la primera versión del sistema. La segunda es conceptual: confirma que la línea seguida en este TFG, basada en integrar modelos con herramientas y fuentes externas, va en la misma dirección que la evolución reciente del ecosistema.

4.2.5 NotebookLM y asistentes centrados en conocimiento aportado por el usuario

NotebookLM [47] representa otra línea de evolución distinta. En lugar de centrarse tanto en la infraestructura o la orquestación, pone el acento en trabajar sobre documentos aportados por el usuario y generar explicaciones, resúmenes y conexiones entre fuentes. Su utilidad como referencia está en recordar que no todos los asistentes documentales buscan resolver el mismo problema.

Frente a NotebookLM, este TFG persigue un objetivo más orientado a entorno corporativo integrado: autenticación, documentos compartidos, actualización periódica, control de acceso y despliegue local. Por tanto, NotebookLM sirve como comparación funcional parcial, no como equivalente arquitectónico.

4.2.6 Frameworks: LangChain y LlamaIndex

LangChain [48] y LlamaIndex [49] son marcos de desarrollo, no productos finales. Ayudan a construir sistemas con LLMs, pero no resuelven por sí solos necesidades como autenticación corporativa, despliegue, observabilidad o interfaz completa.

Por eso, más que competidores directos, deben entenderse como piezas posibles dentro de un sistema mayor.

4.3 Análisis comparativo

La Tabla 4.1 resume la comparación entre la propuesta del TFG y varias alternativas representativas.

Tabla 4.1: Comparación del sistema JARVIS con soluciones alternativas.

Característica	JARVIS	ChatGPT Ent.	Copilot 365	AnythingLLM	Onyx	PrivateGPT	NotebookLM	OpenClaw
On-Premise 100 %	✓	×	×	✓	Parcial	✓	×	✓
Código abierto	✓	×	×	✓	✓	✓	×	✓
UI conversacional	✓	✓	✓	✓	✓	✓	✓	Parcial
RAG documental	✓	Parcial	Parcial	✓	✓	✓	✓	Parcial
SharePoint	✓	×	✓	×	✓	×	×	Parcial
OCR integrado	✓	×	×	×	×	×	×	Parcial
Web / herramientas	✓	Parcial	Parcial	Parcial	Parcial	×	×	✓
SSO corporativo	✓	✓	✓	×	✓	×	×	Parcial
Monitorización propia	✓	×	×	×	Parcial	×	×	Parcial
Lógica extensible	✓	×	×	Limitada	Parcial	Limitada	×	✓

La tabla no debe leerse como una clasificación absoluta, sino como una comparación respecto al objetivo concreto del TFG: asistente corporativo documental, desplegable en local, integrable con SharePoint y extensible mediante lógica propia.

4.4 Posicionamiento del sistema desarrollado

Tras este análisis, el sistema desarrollado ocupa una posición intermedia entre varias familias de soluciones:

1. *Frente a soluciones comerciales*: sacrifica comodidad de servicio gestionado a cambio de soberanía del dato y capacidad de personalización.

2. *Frente a herramientas abiertas listas para usar*: añade más integración corporativa, más control sobre el flujo y más separación clara entre componentes.

3. *Frente a frameworks*: ofrece un sistema desplegable, no solo una base de desarrollo.

4. *Frente a herramientas recientes orientadas a agentes*: comparte la dirección de evolución, pero con un foco más concreto en consulta documental empresarial y trazabilidad.

En resumen, la propuesta de este TFG no compite por ser el asistente más generalista del mercado, sino por resolver de forma sólida un problema muy específico: **consultar documentación corporativa cambiante con lenguaje natural, dentro de una infraestructura controlada y apoyándose en herramientas existentes correctamente integradas**.

Capítulo 5

Descripción Funcional del Sistema

En este capítulo se describe el sistema desde la perspectiva del usuario final. Se detalla qué puede hacer, cómo se presenta la interfaz conversacional, qué funcionalidades están disponibles y cómo puede interactuar el usuario con cada una de ellas. El objetivo es ofrecer una visión completa y comprensible de las capacidades del sistema sin entrar todavía en los detalles de implementación técnica, que se reservan para el Capítulo 6.

5.1 Visión General

JARVIS (*Just A Rather Very Intelligent System*) es un asistente conversacional corporativo desplegado en la infraestructura interna de la organización. Los empleados acceden al sistema a través de un navegador web y pueden interactuar con él en lenguaje natural para realizar las siguientes tareas:

- Consultar documentos corporativos internos (manuales, políticas, procedimientos, informes).
- Analizar páginas web proporcionando una URL.
- Buscar información actualizada en internet.
- Analizar imágenes y documentos escaneados.
- Consultar legislación del Boletín Oficial del Estado (BOE).
- Mantener conversaciones generales con un modelo de lenguaje.
- Consultar el estado de la indexación de documentos.
- Listar los documentos disponibles en la base de conocimiento.

El sistema se ha diseñado para que detecte automáticamente la intención del usuario a partir del texto del mensaje, sin necesidad de comandos explícitos, menús ni modos manuales. El usuario simplemente escribe su pregunta o solicitud y el sistema decide internamente qué funcionalidad activar. Esta transparencia busca que la experiencia sea natural e intuitiva, similar a interactuar con un asistente humano.

5.2 Acceso al Sistema e Inicio de Sesión

El sistema se accede a través de una URL corporativa protegida por HTTPS (puerto 8443). Al acceder por primera vez, el usuario es redirigido al flujo de autenticación corporativa mediante Azure Active Directory (SSO). El proceso es el siguiente:

1. El usuario accede a `https://servidor:8443` desde su navegador.
2. El sistema redirige al portal de autenticación de Azure AD (Microsoft).
3. El usuario introduce sus credenciales corporativas (las mismas que utiliza para acceder a Outlook, Teams o SharePoint).
4. Tras la autenticación exitosa, se le redirige de vuelta al sistema con su sesión iniciada.
5. El sistema identifica automáticamente su nombre, correo electrónico y los grupos a los que pertenece en Azure AD.

No es necesario crear una cuenta separada ni recordar credenciales adicionales. Los grupos del directorio activo determinan a qué documentos tiene acceso cada usuario, implementando así un control de acceso departamental transparente.

5.3 Interfaz de Usuario

La interfaz de usuario es una aplicación web tipo chat, similar en aspecto y funcionamiento a ChatGPT o Microsoft Copilot. En el prototipo implementado, esta capa la proporciona OpenWebUI [26]. La interfaz se estructura en los siguientes elementos principales:

5.3.1 Panel Principal de Chat

El área central de la pantalla constituye el panel de conversación. El usuario escribe su mensaje en un campo de texto en la parte inferior y la respuesta del asistente aparece en la parte superior, en un formato de chat alternado (mensaje del usuario a la derecha, respuesta del asistente a la izquierda).

Las respuestas se muestran en *streaming en tiempo real*: el texto aparece progresivamente, token a token, a medida que el modelo lo genera. Esto proporciona retroalimentación inmediata al usuario y evita la sensación de espera prolongada.

Las respuestas soportan formato enriquecido:

- **Markdown:** Negritas, cursivas, listas, encabezados, código en línea.
- **Bloques de código:** Con resaltado de sintaxis para múltiples lenguajes.
- **Tablas:** Para información estructurada.
- **Enlaces:** URLs clicables a fuentes externas o documentos referenciados.
- *Indicadores visuales*: Etiquetas e iconos que ayudan a identificar el modo de operación activo.

5.3.2 Panel Lateral de Conversaciones

A la izquierda de la interfaz se muestra un panel con el historial de conversaciones del usuario. Cada conversación tiene un título generado automáticamente y puede retomarse en cualquier momento. Las conversaciones se almacenan de forma persistente en la base de datos PostgreSQL del sistema.

5.3.3 Carga de Archivos Adjuntos

La interfaz permite adjuntar archivos directamente en el chat mediante un botón de clip o arrastrando y soltando (*drag and drop*). Los tipos de archivos soportados incluyen:

- **Imágenes** (PNG, JPG, WebP): Se procesan con modelos de visión multimodal para describir, transcribir o analizar su contenido.
- **Documentos PDF:** Si el PDF contiene texto seleccionable, OpenWebUI extrae el texto y lo envía al modelo junto con la pregunta del usuario. Si el PDF es escaneado, se recomienda utilizar la vía de ingesta automatizada (carpeta `data/watch`) para que el motor OCR lo procese.

5.4 Modos de Operación

A continuación se describen los modos de operación fundamentales configurados en el sistema, vistos desde la perspectiva del usuario, con ejemplos de los tipos de consultas que activan cada modo.

5.4.1 Consulta de Documentos Internos (RAG)

Este es el modo principal del sistema y la funcionalidad que justifica todo el proyecto. Cuando el usuario formula una pregunta relacionada con documentos corporativos, el sistema busca automáticamente en su base de conocimiento vectorial y genera una respuesta fundamentada exclusivamente en el contenido de los documentos recuperados.

- **Ejemplo de consulta:** "¿Cuál es la política de calidad de la empresa?"
- **Comportamiento:** El sistema busca en Qdrant los fragmentos más relevantes del documento de calidad indexado, los inyecta como contexto en el prompt y genera una respuesta precisa basada en el contenido real del documento.
- **Indicador visual:** `RAG` `Buscando en documentos internos...`

Las respuestas en modo RAG incluyen *referencias a las fuentes*: el nombre del documento, la página y la puntuación de relevancia de cada fragmento utilizado, proporcionando trazabilidad completa.

El sistema también detecta preguntas de seguimiento: si el usuario pregunta "¿Y cuándo fue la última revisión?" tras una consulta RAG, el sistema mantiene el modo RAG y utiliza el contexto de la conversación anterior para contextualizar la nueva búsqueda.

5.4.2 Búsqueda en Internet

Para consultas que requieren información actualizada o que exceden el ámbito de los documentos internos, el sistema ofrece búsqueda en internet.

- **Ejemplo de consulta:** "Busca en internet las últimas noticias sobre Docker"
- **Comportamiento:** El sistema ejecuta una búsqueda en DuckDuckGo, recupera los resultados más relevantes y genera una respuesta sintetizada con enlaces a las fuentes originales.
- **Indicador visual:** `WEB` `Buscando en internet...`

Las respuestas incluyen los enlaces directos a las fuentes consultadas en una sección de "Fuentes" al final del mensaje. El sistema detecta automáticamente consultas que necesitan información fresca (noticias, precios, resultados deportivos) y añade contexto temporal a la búsqueda.

5.4.3 Análisis de Páginas Web (Scraping)

Cuando el usuario incluye una URL en su mensaje, el sistema descarga y analiza el contenido de la página web automáticamente.

- **Ejemplo de consulta:** "Resume esta página: https://ejemplo.com/articulo"
- **Comportamiento:** El sistema abre la URL en un navegador automatizado (Playwright), espera a que se renderice todo el JavaScript, extrae el contenido textual limpio y genera un resumen o responde la pregunta del usuario basándose en el contenido extraído.
- **Indicador visual:** `URL` `Analizando URL...`

El contenido extraído queda almacenado en la memoria de sesión del usuario, lo que permite realizar preguntas de seguimiento como "¿Qué dice el tercer párrafo?" sin necesidad de volver a enviar la URL. Además, el usuario puede optar por indexar el contenido permanentemente en la colección `webs` de Qdrant para futuras consultas RAG.

5.4.4 Análisis de Imágenes (Visión/OCR)

El sistema permite analizar imágenes adjuntadas directamente en el chat.

- **Ejemplo de consulta:** [Adjuntar foto de un documento] "¿Qué pone en este documento?"
- **Comportamiento:** La imagen se envía a un modelo de visión multimodal (Qwen 2.5 VL en el despliegue actual) que describe, transcribe o analiza el contenido visual.
- **Indicador visual:** `VISION` `Analizando imagen...`

La última imagen analizada queda almacenada en la memoria de sesión, permitiendo preguntas de seguimiento como "¿Y cuál es la fecha que aparece?" sin necesidad de reenviar la imagen.

5.4.5 Consulta Legislativa del BOE

El sistema integra la capacidad de consultar el Boletín Oficial del Estado (BOE) directamente desde la interfaz de chat [50].

- **Ejemplo de consulta:** "Busca en el BOE la ley de protección de datos"
- **Comportamiento:** El sistema resuelve semánticamente el nombre coloquial de la ley a su identificador oficial del BOE, consulta la API de Datos Abiertos del BOE y devuelve un resumen estructurado de la ley solicitada. De forma complementaria, el proyecto incorpora un servidor MCP para este dominio, ya validado fuera de la interfaz principal.
- **Indicador visual:** `BOE` `Consultando el BOE...`

El sistema también soporta consultas por número de artículo ("dame el artículo 5 de la LOPD") y consultas del sumario del día ("¿qué ha salido hoy en el BOE?").

5.4.6 Listar Documentos Disponibles

El usuario puede consultar en cualquier momento qué documentos están indexados en el sistema.

- **Ejemplo de consulta:** "¿Qué documentos tienes?" o simplemente "/docs"
- **Comportamiento:** El sistema consulta las colecciones de Qdrant a las que el usuario tiene acceso y devuelve una lista organizada por tipo (documentos locales y webs guardadas), mostrando el nombre de cada documento.

5.4.7 Estado del Sistema

El usuario puede verificar el estado de la indexación y los procesos internos.

- **Ejemplo de consulta:** "¿Cómo va la indexación?"
- **Comportamiento:** El sistema consulta el servicio Indexer y devuelve información sobre el estado de sincronización con SharePoint, documentos pendientes y estadísticas de la base de conocimiento.

5.4.8 Ayuda y Capacidades

El usuario puede solicitar información sobre las capacidades del sistema.

- **Ejemplo de consulta:** "¿Qué puedes hacer?" o "ayuda"
- **Comportamiento:** El sistema devuelve un mensaje formateado con una descripción de todas las funcionalidades disponibles y ejemplos de uso.

5.4.9 Conversación General

Si el mensaje del usuario no coincide con ninguno de los modos especializados anteriores, el sistema funciona como un asistente conversacional general, respondiendo preguntas, manteniendo conversaciones y proporcionando información a partir del conocimiento general del modelo de lenguaje.

5.5 Escenarios de Uso Completos

Para ilustrar la experiencia del usuario de forma integral, se describen a continuación tres escenarios de uso representativos de las capacidades integradas:

5.5.1 Escenario 1: Consulta de un procedimiento corporativo

Un empleado del departamento de calidad necesita verificar un requisito específico:

1. El empleado accede al sistema e inicia sesión con sus credenciales corporativas.
2. Escribe: "¿Cuáles son los requisitos para la auditoría interna según el manual de calidad?"
3. El sistema detecta la intención RAG, busca en los documentos del departamento de calidad y responde con los requisitos específicos extraídos del manual, incluyendo referencias a la sección y página del documento original.
4. El empleado pregunta: "¿Y cada cuánto tiempo se debe realizar?"
5. El sistema, en modo *sticky*, mantiene el contexto RAG y responde con la periodicidad especificada en el documento.

5.5.2 Escenario 2: Análisis de una normativa del BOE

Un responsable legal necesita revisar una disposición reciente:

1. Escribe: "Busca en el BOE el real decreto sobre teletrabajo"
2. El sistema consulta la API del BOE, localiza el real decreto correspondiente y devuelve un resumen estructurado con título, fecha de publicación, número de artículos y un extracto del contenido.
3. El responsable pregunta: "Dame el artículo 5"
4. El sistema recupera el texto completo del artículo solicitado.

5.5.3 Escenario 3: Análisis de una web tecnológica

Un ingeniero necesita evaluar una herramienta:

1. Escribe: "https://docs.docker.com/compose/compose-file/ ¿Qué opciones de despliegue GPU existen?"
2. El sistema descarga la página, extrae el contenido y genera una respuesta centrada en las opciones de GPU del fichero Docker Compose.
3. El ingeniero pregunta: "Guarda esta web para futuras consultas"
4. El sistema indexa el contenido en la colección `webs` de Qdrant, disponible para consultas RAG posteriores.

5.6 Panel de Administración y Monitorización

Además de la interfaz de chat para usuarios regulares, el sistema ofrece herramientas de administración y monitorización para los administradores del sistema:

5.6.1 Dashboards de Grafana

Los administradores acceden a paneles de control interactivos en Grafana que muestran en tiempo real:

- **Métricas de uso:** Número de consultas por minuto, distribución de modos de operación (cuántas consultas RAG, web search, chat, etc.), tiempos de respuesta (percentiles p50, p95, p99) y tasa de errores.

- **Métricas de GPU:** Consumo de VRAM en tiempo real, temperatura del chip, utilización de los cores CUDA y potencia consumida. Esto permite detectar cuándo la GPU está al límite y ajustar la carga de modelos.
- **Métricas de infraestructura:** Estado de las bases de datos, conexiones activas, espacio disponible en disco.

5.6.2 Administración de la Base de Datos

pgAdmin 4 proporciona acceso directo a la base de datos PostgreSQL del sistema, permitiendo auditar las tablas de usuarios, conversaciones y sesiones almacenadas por OpenWebUI.

5.6.3 Gestión de Modelos

Desde la interfaz de administración de OpenWebUI, los administradores pueden:

- Visualizar los modelos disponibles en Ollama.
- Modificar la configuración de los Pipelines (Valves) sin reiniciar servicios.
- Gestionar usuarios y permisos.

Capítulo 6

Diseño e Implementación

Este capítulo constituye el núcleo técnico de mi Trabajo de Fin de Grado. Aquí describo cómo se ha construido el sistema, qué partes son desarrollo propio y qué partes reutilizan herramientas ya existentes, y cómo se han interconectado los distintos microservicios para convertir una colección de componentes sueltos en un asistente corporativo funcional. Acompaño la explicación con fragmentos de código reales extraídos del repositorio del proyecto.

6.1 Filosofía de Diseño: Necesidad, Solución, Iteración

El sistema no se diseñó como un bloque monolítico, sino que emergió de un proceso iterativo de identificación de necesidades y acoplamiento progresivo de soluciones. Cada nueva funcionalidad surgió como respuesta a una limitación concreta detectada durante el uso del sistema:

1. Necesidad inicial: Un chat con un LLM local. *Solución:* Ollama + OpenWebUI [26], [27]. El sistema comenzó como una instalación básica de OpenWebUI conectada a Ollama, proporcionando una interfaz de chat con modelos locales.

2. Primera limitación: El modelo respondía de memoria, sin acceso a documentos internos. *Solución:* Se desarrolló un backend FastAPI con pipeline RAG y base de datos vectorial Qdrant. Esto exigió un mecanismo para interceptar los mensajes del usuario antes de que llegaran directamente al modelo: el Pipeline de OpenWebUI.

3. Segunda limitación: Los documentos cambiaban frecuentemente en SharePoint y la ingesta manual era inviable. *Solución:* Se creó el servicio Indexer con sincronización automática vía Microsoft Graph API [51].

4. Tercera limitación: Muchos PDFs corporativos eran escaneados y no contenían texto seleccionable. *Solución:* Se integró PaddleOCR con aceleración GPU en el Indexer.

5. Cuarta limitación: Los usuarios necesitaban acceder a información que no estaba en los documentos internos, como noticias, páginas web o normativa. *Solución:* Se añadieron los modos de búsqueda web, scraping y consulta del BOE. En la interfaz principal, la consulta normativa se resuelve mediante API; además, se desarrolló un servidor MCP como línea de integración futura.

6. Quinta limitación: Sin visibilidad sobre el rendimiento del sistema, era imposible detectar cuellos de botella. *Solución:* Se implementó el stack de observabilidad: métricas Prometheus en el backend, NVIDIA DCGM para GPU, y dashboards Grafana [35], [36], [37].

7. Sexta limitación: Las respuestas idénticas saturaban la GPU innecesariamente. *Solución:* Se interpuso LiteLLM como proxy con caché en Redis.

8. Séptima limitación: El modelo no usaba la terminología corporativa adecuada. *Solución:* Se exploró el fine-tuning con LoRA sobre Qwen 2.5.

9. Octava limitación: Los analistas de datos necesitaban consultar tablas de datos estructurados sin aprender SQL. *Solución:* Se desarrolló un agente de traducción NL→SQL (`core/sql_agent.py`) que genera y ejecuta consultas `SELECT` seguras sobre PostgreSQL mediante LiteLLM, con validación por lista blanca y auto-corrección ante errores de sintaxis.

10. Novena limitación: El control de acceso mediante cabeceras HTTP era insuficiente para garantizar la seguridad en entornos multi-tenant con requisitos exigentes. *Solución:* Se implementó validación criptográfica de tokens JWT de Azure AD (`core/auth.py`), con descarga dinámica de claves públicas JWKS, activable con la variable de entorno `AZURE_JWT_VALIDATION=true`.

Esta aproximación iterativa —identificar una carencia, evaluar alternativas, integrar la solución y validar el resultado— se repitió a lo largo de los cinco sprints del proyecto. Cada nueva pieza se acopló al ecosistema respetando la separación de responsabilidades y las interfaces HTTP estándar entre servicios, permitiendo sustituir cualquier componente sin afectar al resto.

6.2 Arquitectura General del Sistema

Para tener visibilidad técnica, se ha diseñado el sistema mediante una arquitectura compuesta por 12 servicios principales y varios servicios auxiliares de soporte y arranque, todos ellos contenerizados con Docker, orquestados mediante Docker Compose [31] y aislados en una red virtual propia (`rag-network`). El flujo resumido de tecnologías ya se ha presentado en el Capítulo 3; aquí se detalla la función de cada servicio dentro de la arquitectura funcional.

Tabla 6.1: Servicios principales de la arquitectura y su función.

Servicio	**Función**	**Puerto**
NGINX	Proxy inverso, terminación TLS	8443 (externo)
OpenWebUI	Interfaz de usuario conversacional	8080 (interno)
Pipelines (JARVIS)	Agente encaminador inteligente	9099 (interno)
Backend (FastAPI)	Motor RAG, scraping, búsqueda, OCR	8000 (interno)
Indexer (FastAPI)	Sincronización SharePoint, ingesta	8001 (interno)
Ollama	Ejecución local de LLMs	11434 (interno)
LiteLLM	Proxy unificado de modelos	4000 (interno)
Qdrant	Base de datos vectorial	6333 (interno)
PostgreSQL	Base de datos relacional	5432 (interno)
Redis	Caché de respuestas	6379 (interno)
MCP-BOE Server	Consulta del BOE	8010 (interno)
MinIO	Almacenamiento de objetos (backup)	9000 (interno)

Todos los servicios comparten una red Docker interna de tipo *bridge* denominada `rag-network`. Únicamente NGINX expone un puerto al exterior (8443), garantizando que ningún servicio interno es accesible directamente desde fuera del servidor.

6.2.1 Orquestación con Docker Compose

La totalidad del ecosistema se define de forma declarativa en un único fichero `docker-compose.yml`. Este manifiesto YAML, alineado con la especificación actual de Compose [31], describe tanto los servicios principales de la arquitectura como varios servicios auxiliares de inicialización y observabilidad, además de sus dependencias (`depends_on`), volúmenes persistentes para datos, variables de entorno, políticas de reinicio (`restart: unless-stopped`) y reservas de recursos GPU.

Los servicios que requieren aceleración GPU (Ollama, PaddleOCR) utilizan la directiva de Docker Compose:

```yaml
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]
```

Listing 6.1: Reserva de GPU en Docker Compose.

Esta configuración, combinada con el `nvidia-container-toolkit` instalado en el host, permite que los contenedores accedan a la GPU del servidor sin necesidad de instalar drivers CUDA dentro de cada contenedor.

Además de las reservas de GPU, el manifiesto define límites de recursos (`deploy.resources.limits`) para los servicios más intensivos, garantizando que ningún servicio monopolice la CPU o la RAM del host en situaciones de carga máxima:

```yaml
# Ejemplos representativos
rag-backend:
  deploy:
    resources:
      limits:
        cpus: "4"
        memory: 8g
qdrant:
  deploy:
    resources:
      limits:
        cpus: "2"
        memory: 4g
redis:
  deploy:
    resources:
      limits:
        memory: 600m
```

Listing 6.2: Límites de recursos por servicio en Docker Compose.

Estos límites protegen la estabilidad del sistema durante picos de ingesta masiva o procesamiento OCR prolongado, evitando que la presión de memoria sobre un servicio degradara al resto.

6.2.2 Flujo de Datos End-to-End

Para ilustrar cómo se conectan las piezas, se describe el recorrido completo de una consulta RAG típica:

1. El **usuario** escribe su pregunta en el navegador (OpenWebUI).
2. **NGINX** recibe la petición HTTPS y la reenvía internamente a OpenWebUI (puerto 8080).
3. **OpenWebUI** envía el mensaje al pipeline JARVIS mediante HTTP POST al puerto 9099.
4. **JARVIS** analiza el texto, detecta la intención RAG, obtiene los departamentos autorizados del usuario y construye una petición al Backend.
5. El **Backend** genera el embedding de la consulta con SentenceTransformer, ejecuta una búsqueda híbrida en Qdrant y recupera los k fragmentos más relevantes.
6. El Backend compone un *prompt* estructurado con el contexto recuperado y lo envía a LiteLLM.
7. **LiteLLM** verifica si existe una respuesta cacheada en Redis. Si no, delega al modelo en Ollama.
8. **Ollama** ejecuta la inferencia en GPU y devuelve los tokens generados en streaming.
9. La respuesta fluye de vuelta: LiteLLM $\rightarrow$ Backend $\rightarrow$ JARVIS $\rightarrow$ OpenWebUI $\rightarrow$ NGINX $\rightarrow$ navegador.

Todo el flujo se completa típicamente en menos de 3.5 segundos, con el primer token visible para el usuario en menos de 1.2 segundos gracias al streaming.

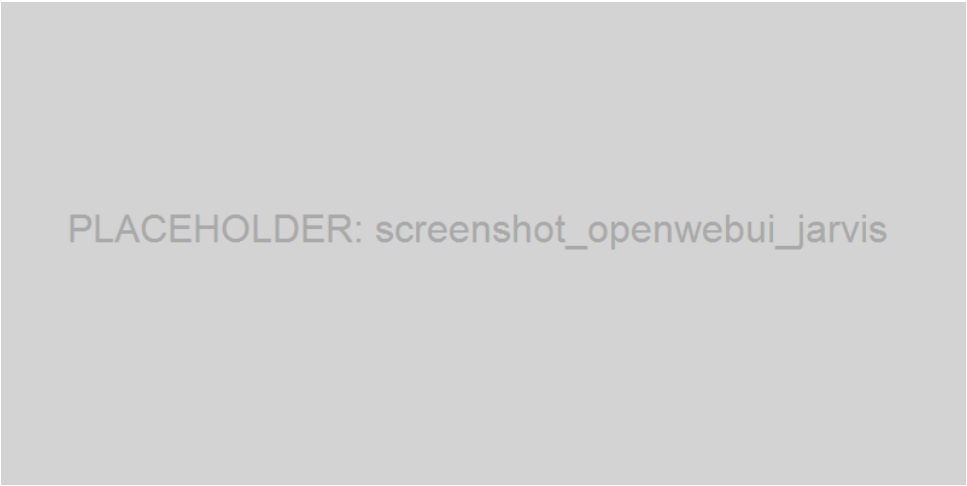

Figura 6.1: Interfaz de OpenWebUI con el modelo JARVIS seleccionado. El usuario escribe su consulta en el campo de texto y recibe la respuesta en streaming.

6.3 Desarrollo Propio vs. Herramientas Externas

Una de las preguntas fundamentales en cualquier proyecto de integración es: ¿qué parte del trabajo es desarrollo original y qué parte es configuración de herramientas existentes? La Tabla 6.3 clasifica cada componente del sistema según su origen.

Tabla 6.2: Resumen sintético de reutilización y desarrollo dentro del TFG.

Componente	Reutilizado	Extendido	Desarrollado en el TFG	Integrado	Papel en el sistema
OpenWebUI	✓	✓	×	✓	Interfaz conversacional web, persistencia de chats y base para autenticación corporativa.
Pipeline JARVIS	×	×	✓	✓	Encaminamiento de intenciones, memoria de sesión y conexión entre interfaz y backend.
Backend RAG	×	×	✓	✓	Recuperación documental, scraping, búsqueda web, API BOE y lógica principal del sistema.
Indexer	×	×	✓	✓	Sincronización con SharePoint, OCR e ingesta automatizada de documentos.
Ollama y LiteLLM	✓	✓	×	✓	Inferencia local, alias de modelos, caché y desacoplo entre servicios y modelos.
Qdrant	✓	✓	×	✓	Almacenamiento vectorial y recuperación documental con filtros por colección.
Servidor MCP-BOE	×	×	✓	✓	Consulta normativa estructurada y prueba de integración MCP.
Prometheus, Grafana y DCGM	✓	✓	×	✓	Observabilidad operativa del backend, la base de datos y la GPU.

La tabla anterior resume de forma deliberadamente simple qué se reutiliza, qué se amplía y qué se implementa desde cero. La tabla siguiente añade más detalle sobre el trabajo concreto realizado en cada pieza.

En resumen, el trabajo de desarrollo propio abarca más de 6.000 líneas de código Python distribuidas entre el Pipeline JARVIS, el Backend, el Indexer y el servidor MCP-BOE, además de toda la configuración declarativa del ecosistema Docker. Las herramientas externas proporcionan las capacidades de infraestructura (base de datos, motor LLM, OCR, navegador), pero la lógica de negocio, la orquestación y la inteligencia del sistema son enteramente originales.

Tabla 6.3: Clasificación de los componentes del sistema según su origen.

Componente	**Tipo**	**Trabajo realizado**
JARVIS Pipeline	**Propio**	1.498 líneas de Python. Detección de intenciones, encaminamiento, memoria de sesión, control de acceso departamental.
Backend FastAPI	**Propio**	3.000 líneas. Pipeline RAG, endpoints REST, chunking, scraping, búsqueda web, métricas Prometheus.
Indexer	**Propio**	1.200 líneas. Sincronización SharePoint, cron interno, ingesta automatizada, detección OCR.
Servidor MCP-BOE	**Propio**	400 líneas. Resolución semántica de leyes, consulta API BOE.
Pipeline Fine-Tuning	**Propio**	Scripts de generación de dataset sintético, entrenamiento LoRA, exportación GGUF.
Docker Compose	**Propio**	Manifiesto de la arquitectura completa, incluyendo servicios principales, redes, volúmenes, GPU y dependencias.
Configuración LiteLLM	**Propio**	YAML de modelos, aliases, caché Redis, fallbacks, rate limits.
Configuración NGINX	**Propio**	Certificados TLS, proxy pass, WebSocket upgrade.
Configuración Prometheus	**Propio**	Targets de scraping, intervalos, reglas.
Dashboards Grafana	**Propio**	3 dashboards diseñados para Backend, GPU y base de datos.
OpenWebUI	**Externo**	Interfaz de usuario. Configurado para SSO, PostgreSQL externo y conexión al Pipeline.
Ollama	**Externo**	Motor de LLM. Configurado con modelos descargados y cuantizados.
Qdrant	**Externo**	Base de datos vectorial. Colecciones y esquemas definidos por el backend propio.
PostgreSQL	**Externo**	Base de datos relacional. Esquema gestionado por OpenWebUI.
Redis	**Externo**	Caché. Configurado con TTL y conectado a LiteLLM.
PaddleOCR	**Externo**	Motor OCR. Integrado dentro del Indexer propio con fallback GPU/CPU.
Playwright	**Externo**	Automatización de navegador. Integrado en el backend propio.
MinIO	**Externo**	Almacenamiento S3. Configurado para backup de documentos.
Microsoft Graph API	**Integración**	Autenticación MSAL, sincronización delta de SharePoint.
Azure AD (OIDC)	**Integración**	SSO corporativo, extracción de grupos para control de acceso.
API BOE	**Integración**	Consulta del Boletín Oficial del Estado.
DuckDuckGo	**Integración**	Búsqueda en internet con fallback HTML.

6.4 El Mecanismo Pipeline de OpenWebUI (Hook)

El Pipeline de OpenWebUI es el punto de conexión fundamental entre la interfaz de usuario y todo el backend del sistema [26]. Sin este mecanismo, OpenWebUI sería simplemente una interfaz para chatear directamente con Ollama. El Pipeline es lo que convierte esa interfaz genérica en un asistente corporativo inteligente.

6.4.1 Concepto de Hook

En ingeniería de software, un *hook* es un punto de extensión que permite inyectar código personalizado en un flujo de ejecución predefinido sin modificar el sistema original. En el caso de OpenWebUI, el *hook* se materializa en el sistema de Pipelines: un mecanismo que permite interponer un servicio externo entre el usuario y el modelo de lenguaje.

Cuando un usuario envía un mensaje en OpenWebUI, en lugar de reenviarlo directamente a Ollama, OpenWebUI lo envía primero al servicio Pipeline configurado. El Pipeline procesa el mensaje, ejecuta la lógica necesaria y devuelve la respuesta final al usuario. Desde la perspectiva de OpenWebUI, el Pipeline *es* el modelo.

6.4.2 Descubrimiento y Conexión del Pipeline

La conexión entre OpenWebUI y el Pipeline se establece mediante configuración:

1. El Pipeline se despliega como un contenedor Docker que expone una API HTTP en el puerto 9099.

2. OpenWebUI se configura con la variable de entorno `OPENAI_API_BASE_URLS` apuntando al servicio Pipeline (`http://pipelines:9099`).

3. Al arrancar, OpenWebUI descubre el Pipeline por su nombre (`JARVIS`) y lo muestra como un modelo seleccionable en la interfaz.

4. Cuando el usuario selecciona el modelo "JARVIS" y envía un mensaje, OpenWebUI realiza un HTTP POST al Pipeline con el mensaje, el historial de conversación y los metadatos del usuario.

6.4.3 Contrato de la Clase Pipeline

El Pipeline se implementa como una clase Python que debe cumplir un contrato específico:

```
class Pipeline:
    class Valves(BaseModel):
        BACKEND_URL: str = "http://rag-backend:8000"
        LITELLM_URL: str = "http://litellm:4000"
        TEXT_MODEL: str = "JARVIS"
        VISION_MODEL: str = "qwen2.5vl"
        DEPARTMENT_MAPPING: Dict[str, str] = {
            "CIVEX2": "documents_CIVEX2",
        }
        DEFAULT_COLLECTION: str = "documents"

    def __init__(self):
        self.name = "JARVIS"
        self.id = "jarvis"
        self.valves = self.Valves()
        # Memoria de sesion por usuario
        self._user_image_memory: Dict[str, str] = {}
        self._user_web_memory: Dict[str, Dict] = {}

    async def on_startup(self):
        logger.info("Starting JARVIS")

    async def on_shutdown(self):
        logger.info("Shutting down JARVIS")

    def pipe(self, user_message, model_id, messages, body):
        # HOOK PRINCIPAL: toda la logica del sistema
        intent = self._detect_intent(user_message, body, messages)
        action = intent["action"]
        # ... encaminamiento a subsistemas ...
```

Listing 6.3: Estructura del Pipeline JARVIS.

6.4.4 El Método `pipe()`: Corazón del Sistema

El método `pipe()` es el punto de entrada de toda la lógica del sistema. Recibe cuatro parámetros:

- `user_message` (`str`): El texto del mensaje del usuario.
- `model_id` (`str`): El identificador del modelo seleccionado ("jarvis").

- `messages` (`List[Dict]`): El historial completo de la conversación actual, con mensajes del usuario y del asistente.
- `body` (`Dict`): Metadatos de la petición, incluyendo el objeto `user` con email, nombre, rol y grupos del usuario autenticado, así como archivos e imágenes adjuntos.

El método utiliza la palabra clave `yield` de Python para devolver la respuesta en modo streaming. En lugar de esperar a que se genere la respuesta completa, el Pipeline va devolviendo fragmentos de texto a medida que están disponibles. OpenWebUI recibe estos fragmentos y los muestra progresivamente en la interfaz:

```python
def pipe(self, user_message, model_id, messages, body):
    intent = self._detect_intent(user_message, body, messages)
    action = intent["action"]

    if action == "rag":
        yield "Buscando en documentos internos...\n\n"
        response = self._call_backend_chat(
            user_message, "rag", user_data
        )
        yield response["content"]
        yield "\n\n---\n### Fuentes:\n"
        for source in response["sources"]:
            yield f"- {source['filename']} (p.{source['page']})\n"

    elif action == "web_search":
        yield "Buscando en internet...\n\n"
        # ... logica de busqueda web ...

    elif action == "chat":
        # Conversacion directa con LLM
        # ...
```

Listing 6.4: Ejemplo de respuesta streaming en el Pipeline.

6.4.5 Sistema Valves: Configuración Dinámica

La clase anidada `Valves` hereda de `Pydantic BaseModel` y define todos los parámetros configurables del Pipeline. La particularidad es que OpenWebUI detecta automáticamente esta clase y expone sus campos como formulario editable en la interfaz de administración. Esto significa que un administrador puede modificar la URL del backend, cambiar el modelo de texto, añadir departamentos al mapeo o activar/desactivar el modo de depuración directamente desde el navegador, sin editar código ni reiniciar servicios.

Los parámetros configurados como Valves incluyen:

- `BACKEND_URL`: URL del servicio Backend FastAPI.
- `LITELLM_URL`: URL del proxy LiteLLM.
- `TEXT_MODEL`: Modelo de texto por defecto.
- `VISION_MODEL`: Modelo de visión para análisis de imágenes.
- `DEPARTMENT_MAPPING`: Diccionario que mapea grupos de Azure AD a colecciones de Qdrant.
- `DEFAULT_COLLECTION`: Colección por defecto para usuarios sin grupo específico.
- `PERMISSION_CACHE_TTL`: Tiempo de vida de la caché de permisos.
- `INDEXER_URL`: URL del servicio Indexer para consultas de estado.
- `DEBUG_MODE`: Activar/desactivar logs detallados.

6.5 Agente Encaminador JARVIS: Inteligencia Central

El agente JARVIS es la pieza de software más compleja del proyecto, con 1.498 líneas de código Python. Su función es analizar cada mensaje del usuario y decidir a cuál de los 10 modos de operación derivarlo.

6.5.1 Sistema de Detección de Intenciones

La detección de intenciones se implementa en el método `_detect_intent()`, que evalúa el mensaje del usuario contra un conjunto de reglas heurísticas basadas en coincidencia de palabras clave y expresiones regulares. Las reglas se evalúan en un *orden de prioridad estricto*: las condiciones más específicas se verifican primero (presencia de archivos adjuntos, imágenes, URLs) y se desciende progresivamente hacia las heurísticas más genéricas (keywords de RAG, búsqueda web, conversación general).

La Tabla 6.4 resume las categorías de intención, su prioridad y los criterios de activación.

Tabla 6.4: Categorías de intención y su prioridad de evaluación.

Prioridad	**Modo**	**Criterio de activación**
0	Decisión pendiente	Existe una decisión de URL esperando respuesta del usuario
1	Archivo adjunto	Se detecta un PDF o documento en el cuerpo de la petición
2	Visión/OCR	Imagen adjunta en el mensaje actual
3	Referencia a imagen	Mención a "la imagen" con imagen previa en memoria
4	Referencia a web	Mención a "la web" o "la página" con contenido web en memoria
5	Ayuda	Keywords: "qué puedes hacer", "ayuda", "help"
6	Listar documentos	Commands: "/docs", "/listar"; keywords: "qué documentos tienes"
7	BOE / Legal	Keywords: "busca en el BOE", "ley de", "real decreto"
8	RAG Documental	Keywords: "documento", "política", "manual", "procedimiento"
9	URL detectada	Expresión regular que detecta `https://...` en el mensaje
10	Búsqueda web	Keywords: "busca en internet", "noticias", "precio"
11	Estado del sistema	Keywords: "cómo va", "estado", "indexación"
12	Sticky Mode	Seguimiento contextual del modo de la pregunta anterior
13	Chat general	Default: cualquier mensaje que no active otro modo

Figura 6.2: Ejemplo de respuesta RAG con fuentes documentales. El sistema muestra el contenido recuperado junto con las referencias a los documentos y páginas de origen.

6.5.2 Sticky Mode: Persistencia de Contexto

Una característica avanzada del encaminador es el *Sticky Mode*: cuando un usuario está inmerso en una conversación RAG y formula una pregunta de seguimiento breve ("¿Y las fechas?", "Dime más"), el sistema detecta que la pregunta anterior era de tipo RAG y mantiene el mismo modo, evitando que keywords ambiguos como "fechas" o "resultados" desvíen la consulta al modo de búsqueda web.

```
# Si el mensaje es corto y ambiguo,
# mirar de que hablaba la pregunta anterior
if msg_len < 25 and is_ambiguous and messages:
    last_user_msg = messages[-2].get("content", "").lower()
    was_rag = any(kw in last_user_msg for kw in rag_keywords)
    if was_rag:
        logger.info("Sticky Mode: Reusing RAG intent")
        return {"action": "rag", "metadata": {}}
```

Listing 6.5: Implementación del Sticky Mode.

6.5.3 Memoria de Sesión por Usuario

JARVIS mantiene tres estructuras de estado en memoria para cada usuario, identificado por su dirección de correo electrónico:

- `_user_image_memory`: Almacena la última imagen enviada en formato Base64, permitiendo preguntas de seguimiento sin reenviar la imagen.

- `_user_web_memory`: Retiene el contenido extraído de la última URL analizada, habilitando consultas contextuales sobre la página.
- `_user_pending_decision`: Gestiona decisiones pendientes cuando el sistema detecta que una URL ya fue indexada previamente y ofrece al usuario la opción de reutilizar el contenido existente o volver a scrapearla.

6.5.4 Control de Acceso Departamental

El Pipeline integra un sistema de permisos multi-departamento que segmenta el acceso a las colecciones vectoriales según los grupos de Azure Active Directory del usuario. El flujo es:

1. Al recibir una petición, JARVIS extrae los grupos del objeto `user_data` proporcionado por OpenWebUI.
2. Si los grupos no están disponibles en `user_data`, JARVIS consulta directamente la API de Microsoft Graph para obtener los grupos del usuario mediante el flujo de credenciales de cliente.
3. Los grupos se mapean a colecciones de Qdrant mediante el diccionario configurable `DEPARTMENT_MAPPING`.
4. Las colecciones autorizadas se envían al Backend en la cabecera HTTP `X-Tenant-Ids`, que filtra la búsqueda vectorial a solo los documentos permitidos.
5. Los permisos se cachean durante 300 segundos (configurable) para minimizar llamadas a Graph API.

6.6 Backend RAG: Motor de Búsqueda Semántica

El Backend constituye el motor central de procesamiento del sistema. Implementado en FastAPI, fue objeto de una refactorización significativa durante el desarrollo: inicialmente era un único fichero `main.py` de más de 1.900 líneas que se dividió en una arquitectura modular con paquetes separados para esquemas (`schemas/`), servicios (`services/`), endpoints (`api/`), procesamiento (`processing/`) e integraciones (`integrations/`).

6.6.1 Pipeline de Ingesta Documental

El proceso de ingesta de un documento sigue las siguientes etapas:

1. Recepción: El documento llega al backend vía API REST (carga manual), vía Indexer (sincronización SharePoint) o vía scraper (extracción web).

2. Detección de tipo: Se intenta extraer texto directamente del PDF con `pdfplumber`. Si el texto extraído es insuficiente (< 100 caracteres por página), se asume que el documento es escaneado y se redirige al motor OCR.

3. Extracción de texto: Para PDFs nativos se usa `pdfplumber`; para escaneados, PaddleOCR con aceleración GPU; para HTML, la cadena Trafilatura $\rightarrow$ Readability $\rightarrow$ BeautifulSoup.

4. Chunking semántico: El texto extraído se segmenta en fragmentos de 512–1024 tokens con un solapamiento del 10–15 % entre fragmentos consecutivos. El solapamiento es fundamental para no perder el hilo semántico de párrafos que cruzan los límites de ventana.

5. Generación de embeddings: Cada fragmento se transforma en un vector denso de 384 dimensiones mediante el modelo `paraphrase-multilingual-MiniLM-L12-v2` de SentenceTransformers, ejecutado en CPU para evitar conflictos de VRAM con los modelos de generación.

6. Almacenamiento en Qdrant: Los vectores se almacenan en la colección correspondiente de Qdrant junto con un payload de metadatos que incluye el texto original, nombre de fichero, página, índice de chunk, fuente, marca temporal y tenant_id.

6.6.2 Búsqueda Híbrida

La recuperación de contexto para consultas RAG combina dos estrategias complementarias:

1. Búsqueda densa (HNSW): La consulta del usuario se transforma en un embedding de 384 dimensiones y se buscan los fragmentos más próximos en el espacio vectorial mediante el índice HNSW de Qdrant.

2. Búsqueda dispersa (BM25): Se complementa con búsqueda léxica que favorece coincidencias exactas de términos. Esto es especialmente útil para identificadores, códigos de documento, siglas o artículos legales que la búsqueda semántica podría no capturar con precisión.

Los resultados de ambas búsquedas se combinan y se reordenan para seleccionar los k fragmentos más relevantes (típicamente $k = 5$).

6.6.3 Construcción del Prompt Contextual

El prompt que se envía al LLM se estructura en tres partes:

```
system_prompt = """Eres un asistente corporativo.
Responde UNICAMENTE basandote en el contexto proporcionado.
Si la informacion no esta en el contexto, di que no la tienes.
NO inventes informacion."""

context = "\n\n".join([
    f"[{r.filename} | pagina {r.page} | score={r.score:.2f}]\n{r.text}"
    for r in search_results
])

user_prompt = f"CONTEXTO:\n{context}\n\nPREGUNTA:\n{query}"
```

Listing 6.6: Estructura del prompt RAG.

El *system prompt* restrictivo, combinado con una temperatura de inferencia baja (0,1–0,2), elimina prácticamente las alucinaciones en modo RAG: el modelo solo puede responder con información que esté presente en los fragmentos recuperados.

6.7 Extracción Web: Scraping y Búsqueda en Internet

6.7.1 Motor de Renderizado con Playwright

El módulo de scraping utiliza Playwright para instanciar un navegador Chromium completo en modo *headless*. A diferencia de soluciones basadas en peticiones HTTP simples (como `requests`), este enfoque permite:

- Renderizar páginas JavaScript complejas (React, Vue, Angular).
- Ejecutar la carga diferida (*lazy loading*) de contenido.
- Resolver redirecciones y consentimientos de cookies.
- Capturar el DOM final tras la ejecución de todos los scripts.

El scraper incorpora rotación de *User-Agent*, reintentos con retroceso exponencial (*exponential backoff*) y una espera inteligente configurable para aplicaciones de una sola página (SPA).

6.7.2 Cadena de Extracción con Fallback

Tras obtener el HTML renderizado, se aplica una cadena de extracción compuesta por tres estrategias de *fallback*:

1. Trafilatura: Algoritmo NLP que extrae el contenido principal eliminando menús, cabeceras y publicidad. Se prioriza por su alta precisión.
2. Readability: Algoritmo basado en heurísticas de densidad de texto, comparable al modo lectura de los navegadores.
3. BeautifulSoup: Extracción cruda de todo el texto visible del DOM como última alternativa si las dos anteriores producen menos de 200 caracteres.

6.7.3 Búsqueda en Internet

El modo de búsqueda web utiliza la librería `duckduckgo_search` con un *fallback* propio que realiza scraping HTML directo sobre el dominio `html.duckduckgo.com`. El sistema detecta automáticamente consultas que necesitan información fresca (noticias, precios, eventos) y enriquece el query con el año actual para obtener resultados actualizados.

6.8 Reconocimiento Óptico de Caracteres (OCR)

El motor OCR PaddleOCR se integra en el servicio Indexer y se ejecuta con aceleración GPU. La inicialización implementa un mecanismo de *fallback* automático:

```
try:
    self.ocr = PaddleOCR(
        use_angle_cls=True, lang="latin",
        use_gpu=True, gpu_mem=500
    )
except Exception:
    logger.warning("GPU no disponible, modo CPU")
    self.ocr = PaddleOCR(
        use_angle_cls=True, lang="latin",
        use_gpu=False
    )
```

Listing 6.7: Inicialización de PaddleOCR con fallback GPU/CPU.

Si la GPU no está disponible (fallo de driver, VRAM agotada), el motor se reinicia automáticamente en modo CPU sin intervención manual, garantizando la continuidad del servicio de ingesta.

6.9 Integración con Microsoft SharePoint

El módulo Indexer funciona como un servicio autónomo (*worker*) que ejecuta un ciclo de sincronización periódica cada 5 minutos.

6.9.1 Autenticación MSAL

La conexión con SharePoint se realiza mediante OAuth 2.0 con flujo de credenciales de cliente (*Client Credentials Flow*). El servicio obtiene un token de acceso del *tenant* de Azure AD mediante la librería MSAL y utiliza la API REST de Microsoft Graph [51] para listar y descargar ficheros.

6.9.2 Sincronización Delta Incremental

Para evitar reprocesar documentos ya ingeridos, el Indexer implementa sincronización delta: consulta la marca temporal de modificación de cada fichero en SharePoint y la compara con los registros almacenados en PostgreSQL. Solo los ficheros nuevos o modificados se descargan, procesan e indexan. Esta estrategia reduce drásticamente el consumo de red y GPU en ciclos de sincronización recurrentes.

6.10 Integración con el Boletín Oficial del Estado (BOE)

La integración con el BOE se implementa mediante un servidor MCP (*Model Context Protocol*) construido con la librería `fastmcp`. El servidor expone funciones de búsqueda y consulta que transforman peticiones en lenguaje natural en llamadas estructuradas a la API de Datos Abiertos del BOE.

Esta integración no es una búsqueda web genérica, y merece una justificación específica. En el contexto del tipo de empresa para la que se diseña este sistema, el acceso a normativa legal (legislación laboral, protección de datos, contratación pública) es frecuente y tiene consecuencias directas sobre decisiones operativas. Un error en la interpretación de una norma no es solo un fallo académico: puede traducirse en incumplimientos, sanciones o decisiones mal fundamentadas.

El BOE presenta características que lo distinguen como caso de uso propio:

- Las leyes se identifican por códigos oficiales (p. ej. `BOE-A-2018-16673`), no por títulos que el usuario recuerde fielmente.
- Un mismo texto legal puede referenciarse coloquialmente de docenas de formas distintas (“la LOPD”, “la ley de privacidad”, “el reglamento de protección de datos”).

- La información relevante suele estar en artículos numerados concretos, no en un fragmento genérico.
- Tanto la exactitud como la fuente de procedencia son cruciales: citar una norma derogada o una sección incorrecta tiene consecuencias.

Por todo ello se implementó una capa específica de resolución semántica y consulta estructurada —en lugar de delegar estas peticiones a un buscador generalista— que traduce el lenguaje del usuario al identificador normativo correcto antes de invocar la API oficial.

6.10.1 Resolución Semántica de Leyes

Para consultas como "Resume la Ley de Protección de Datos", el sistema implementa un mecanismo que mapea nombres coloquiales a identificadores oficiales del BOE:

```
LAW_MAPPINGS = {
    "proteccion de datos": "BOE-A-2018-16673",
    "constitucion": "BOE-A-1978-31229",
    "propiedad intelectual": "BOE-A-1996-8930",
    "igualdad": "BOE-A-2007-6115",
}
```

Listing 6.8: Mapeo semántico de nombres de leyes.

El resolvedor elimina palabras vacías comunes del español ("la", "de", "del", "ley") y busca coincidencias parciales en el diccionario antes de acudir al endpoint de búsqueda general del BOE.

En el alcance actual, el servidor MCP se ha validado en ejecución por terminal con un LLM y un cliente MCP compatible. Aunque OpenWebUI ya incorpora soporte MCP en versiones recientes [25], la funcionalidad BOE dentro de la interfaz web se integra aquí mediante llamadas a API directas desde el pipeline JARVIS. Esta decisión permitió mantener un mayor control sobre autenticación, encaminamiento y trazabilidad durante el desarrollo, dejando preparada la migración a MCP como evolución posterior.

6.11 Seguridad Perimetral y Autenticación

6.11.1 NGINX como Proxy Inverso

NGINX actúa como la primera barrera de seguridad y el único servicio expuesto al exterior. Su configuración incluye:

- Terminación TLS: Todo el tráfico entre el navegador y el servidor se cifra con un certificado TLS autofirmado. Los microservicios internos se comunican en texto plano dentro de la red Docker aislada.
- Proxy pass transparente: Las peticiones HTTPS se redirigen a OpenWebUI (puerto 8080) preservando las cabeceras `X-Forwarded-For` y `X-Real-IP` para la trazabilidad.
- WebSocket upgrade: Se habilita la mejora de protocolo a WebSocket, requisito indispensable para el streaming de tokens en tiempo real.

6.11.2 SSO con Azure Active Directory

OpenWebUI se configura con el protocolo OpenID Connect (OIDC) apuntando al *tenant* corporativo de Azure AD. Los usuarios se autentican con sus credenciales corporativas sin crear cuenta separada. Los tokens JWT emitidos por Azure AD se validan en cada petición y sus atributos (nombre, correo, grupos) se utilizan para el control de acceso departamental en JARVIS.

6.11.3 Validación JWT en el Backend

Además de la autenticación perimetral de OpenWebUI vía OIDC, el Backend implementa una segunda capa de validación directa de tokens JWT para clientes externos y herramientas MCP que llaman a los endpoints sin pasar por la interfaz web. La lógica, implementada en `backend/app/core/auth.py`, sigue el flujo estándar de validación de tokens Bearer de Azure AD:

1. El backend descarga dinámicamente el conjunto de claves públicas del *tenant* corporativo desde el endpoint JWKS de Azure AD.
2. Las claves se cachean en memoria para evitar latencia en peticiones sucesivas.
3. Cada token JWT recibido en la cabecera `Authorization: Bearer` se verifica criptográficamente contra esas claves.
4. Los grupos del usuario extraídos del token se mapean a colecciones de Qdrant autorizadas.

5. Si el token no es válido o el usuario carece de acceso a alguna colección, el backend devuelve `HTTP 403 Forbidden` antes de ejecutar ninguna búsqueda.

Esta funcionalidad se activa con la variable de entorno `AZURE_JWT_VALIDATION=true` y es completamente transparente para los clientes que ya utilizan el flujo SSO de OpenWebUI.

6.11.4 Aislamiento de Red

Toda la malla de servicios convive dentro de un plano interno `rag-network`, una red Docker de tipo *bridge*. Esto significa que servicios como PostgreSQL (5432), Redis (6379), Qdrant (6333) y Ollama (11434) no son accesibles desde fuera del servidor bajo ninguna circunstancia, eliminando vectores de ataque sobre estas bases de datos.

6.12 Stack de Observabilidad: Prometheus y Grafana

La monitorización del sistema fue una funcionalidad implementada en respuesta a una necesidad real: durante las primeras fases de desarrollo, no existía visibilidad sobre el rendimiento del sistema, la saturación de la GPU ni los patrones de uso. Sin métricas, la optimización era intuitiva en lugar de basada en datos. Para resolverlo se adoptó un stack apoyado en Prometheus, Grafana y NVIDIA DCGM Exporter [35], [36], [37].

6.12.1 Métricas Personalizadas en el Backend

El backend FastAPI expone métricas en formato Prometheus a través del endpoint `/metrics`. Las métricas implementadas incluyen:

- Contador de peticiones HTTP: Total de peticiones por endpoint, método y código de respuesta.
- Histograma de latencia: Distribución de tiempos de respuesta por endpoint, permitiendo calcular percentiles (p50, p95, p99).
- Distribución de modos: Contador de cuántas consultas se procesan en cada modo (RAG, chat, web search, scraping, OCR, BOE).
- Errores: Tasa de errores por tipo (timeout, modelo no disponible, Qdrant inalcanzable).

6.12.2 Telemetría GPU con NVIDIA DCGM Exporter

NVIDIA DCGM Exporter captura métricas de rendimiento de la GPU del servidor y las expone en formato Prometheus. Las métricas incluyen:

- Consumo de VRAM (memoria GDDR6 utilizada vs. total).
- Utilización de CUDA cores (porcentaje de uso del procesador gráfico).
- Temperatura del chip GPU en grados Celsius.
- Potencia consumida en vatios.

Estas métricas resultaron críticas para optimizar la gestión de modelos: la observación del consumo de VRAM en tiempo real permitió determinar qué combinaciones de modelos podían coexistir simultáneamente en los 24 GB de VRAM disponibles y cuándo era necesario recurrir a cuantizaciones más agresivas (Q4 en lugar de Q8).

6.12.3 Diseño de Dashboards en Grafana

Se diseñaron y configuraron tres dashboards interactivos en Grafana:

1. Dashboard de Backend: Panel principal con latencia media y percentiles, throughput (peticiones/minuto), tasa de errores, distribución de modos de operación en tiempo real, y un gráfico de tendencia de uso a lo largo del tiempo.
2. Dashboard de GPU: Visualización en tiempo real del consumo de VRAM con alarmas de umbral (80 % de capacidad), temperatura del chip con evolución temporal, utilización de cores y potencia. Este dashboard permitió identificar que la coexistencia de modelos grandes de la familia Qwen 2.5 debía gestionarse con carga bajo demanda en Ollama para no comprometer el resto de servicios.
3. Dashboard de Base de Datos: Métricas de PostgreSQL (conexiones activas, queries por segundo) y estadísticas de Qdrant (número de puntos indexados, latencia de búsqueda).

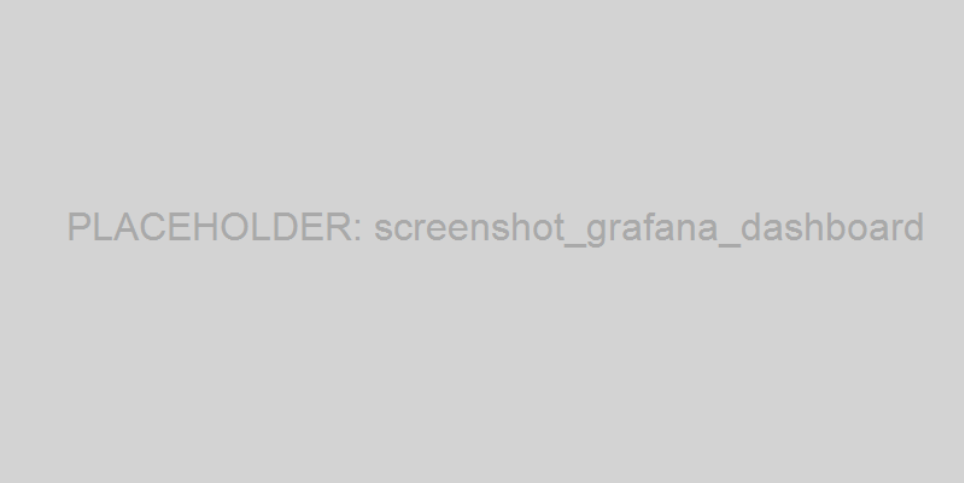

Figura 6.3: Dashboard de Grafana mostrando las métricas del backend en tiempo real: latencia, distribución de modos de operación y tasa de errores.

6.12.4 Impacto de la Monitorización

La implementación del stack de observabilidad no fue un ejercicio académico: proporcionó información que condujo directamente a decisiones de optimización:

- Se identificó que el 35 % de las consultas repetidas se beneficiaban de la caché Redis, validando la decisión de interponer LiteLLM.
- Se detectó que las consultas RAG con más de 5 fragmentos de contexto no mejoraban la calidad de la respuesta pero aumentaban significativamente la latencia, optimizando el valor de k.
- Se observó que PaddleOCR monopolizaba la GPU durante los ciclos de ingesta, lo que motivó la programación de la ingesta en horarios de baja actividad.

6.13 Caché y Rendimiento: Redis

Redis actúa como capa de caché integrada con LiteLLM. Su configuración es directa: LiteLLM almacena en Redis un hash de cada consulta (prompt + parámetros del modelo) junto con la respuesta generada. Las siguientes consultas idénticas se resuelven directamente desde la caché sin invocar a Ollama, eliminando la latencia de inferencia GPU.

Cada entrada tiene un TTL (*Time To Live*) configurable de 1 hora, tras el cual se invalida automáticamente. La tasa de aciertos de caché (*cache hit rate*) observada en producción fue de aproximadamente el 35 %, lo que representó una reducción significativa de la carga sobre la GPU.

Adicionalmente, el pipeline JARVIS utiliza su propio mecanismo de caché en memoria para los permisos departamentales, evitando consultas repetitivas a Microsoft Graph API. Este caché tiene un TTL independiente de 300 segundos.

6.14 Almacenamiento de Objetos: MinIO

MinIO proporciona un sistema de almacenamiento de objetos compatible con Amazon S3 dentro del ecosistema. Su función principal es preservar una réplica aislada de todos los documentos binarios ingeridos por el sistema, proporcionando un mecanismo de backup independiente que no depende de la base de datos vectorial ni de la relacional.

En caso de que Qdrant pierda datos o se requiera una reindexación completa, los documentos originales pueden recuperarse desde MinIO y reprocesarse sin necesidad de acceder nuevamente a las fuentes originales (SharePoint, web).

6.15 Agente SQL: Consultas en Lenguaje Natural sobre Datos Estructurados

Una limitación del pipeline RAG estándar es que opera exclusivamente sobre texto no estructurado: documentos, páginas web y fragmentos de texto. Sin embargo, muchos sistemas empresariales almacenan información crítica en tablas relacionales: métricas de uso, inventarios, datos financieros o registros de operaciones. Para cubrir esta necesidad sin requerir que los usuarios aprendan SQL, se desarrolló el agente SQL.

6.15.1 Arquitectura del Agente NL→SQL

El agente, implementado en `backend/app/core/sql_agent.py`, sigue un flujo de tres pasos:

1. **Inspección del esquema:** Al inicializarse, el agente consulta el catálogo de PostgreSQL para obtener la estructura de las tablas autorizadas (columnas, tipos y restricciones), construyendo una representación textual del esquema que se inyecta como contexto al LLM.

2. **Generación de SQL:** La pregunta del usuario, junto con el esquema, se envía a LiteLLM para generar una consulta `SELECT`. El *prompt* instruye explícitamente al modelo a generar solo consultas de lectura, sin `INSERT`, `UPDATE`, `DELETE` ni `DROP`.

3. **Ejecución y auto-corrección:** La consulta generada se valida contra una lista blanca de operaciones permitidas. Si la ejecución produce un error de sintaxis, el agente reintenta con el mensaje de error como contexto adicional, hasta un máximo de dos reintentos antes de informar al usuario del fallo.

6.15.2 Mecanismos de Seguridad

El acceso no autorizado o accidental a datos sensibles se previene mediante varias capas de seguridad:

```
ALLOWED_PATTERN = re.compile(
    r"^\s*(SELECT|WITH|EXPLAIN)\s",
    re.IGNORECASE
)

def _validate_query(self, sql: str) -> bool:
    if not ALLOWED_PATTERN.match(sql.strip()):
        raise SecurityError("Solo se permiten consultas SELECT")
    for forbidden in ["DROP", "DELETE", "INSERT",
                      "UPDATE", "TRUNCATE"]:
        if forbidden in sql.upper():
            raise SecurityError(
                f"Operacion prohibida detectada: {forbidden}"
            )
    return True
```

Listing 6.9: Validación de la consulta generada contra lista blanca.

Adicionalmente, las consultas se ejecutan con un *timeout* configurable para prevenir consultas costosas que saturen la base de datos.

6.15.3 Endpoint Unificado RAG+SQL

El módulo `backend/app/api/query.py` expone un endpoint unificado `POST /api/v1/query` que enruta automáticamente entre el pipeline RAG y el agente SQL según la naturaleza de la consulta. El enrutamiento se basa en expresiones regulares que detectan patrones propios de consultas sobre datos estructurados ("cuántos", "total de", "lista de registros", "promedio de") frente a patrones de búsqueda documental. Este diseño es transparente para el usuario: la misma interfaz resuelve tanto preguntas sobre documentos como preguntas sobre datos tabulares.

6.16 Fine-Tuning con LoRA

6.16.1 Generación del Dataset de Entrenamiento

Para adaptar el modelo base al dominio corporativo, se implementó un pipeline de generación automática de datos de entrenamiento. Un script extrae pares pregunta-respuesta directamente de los fragmentos indexados en Qdrant, utilizando un LLM base local como generador sintético de consultas plausibles.

6.16.2 Entrenamiento con Low-Rank Adaptation

El ajuste fino se realizó mediante LoRA, que inserta matrices de rango reducido ($r = 8$) en las capas de atención del transformador sin modificar los pesos originales:

$$W' = W + \Delta W = W + BA \tag{6.1}$$

donde $A \in \mathbb{R}^{r \times d}$ y $B \in \mathbb{R}^{k \times r}$ con $r \ll \text{mín}(d, k)$. Esto reduce el número de parámetros entrenables de millones a miles, haciendo viable el ajuste fino con una única GPU de consumo.

6.16.3 Resultado y Evaluación

El modelo resultante, `rag-qwen-ft:latest`, basado en Qwen 2.5, se registró en LiteLLM como el modelo primario bajo el alias "JARVIS" para consultas RAG corporativas.

No obstante, se observó empíricamente que para tareas de RAG puras, el *fine-tuning* del LLM resultaba menos eficaz que un buen ajuste del prompt (*prompt engineering*) combinado con un modelo de embeddings bien adaptado al dominio. La estrategia final del sistema descansa más sobre la calidad del contexto recuperado (chunking, embeddings, búsqueda híbrida, prompt restrictivo) que sobre la modificación de los pesos del modelo generativo.

Capítulo 7

Metodología

En este capítulo se describe el proceso de desarrollo seguido, el entorno hardware utilizado, los criterios de selección tecnológica y la estrategia de documentación adoptada. El objetivo es mostrar cómo se ha organizado el trabajo y con qué medios se ha llevado a cabo.

7.1 Proceso de Desarrollo

El diseño e implementación de este Trabajo de Fin de Grado se ha conducido siguiendo un marco de trabajo ágil (*Agile*), caracterizado por entregas iterativas e incrementales. Dada la naturaleza cambiante del ámbito de la Inteligencia Artificial y de los Modelos de Lenguaje Grande (LLMs), la flexibilidad para revisar decisiones técnicas, sustituir herramientas o ajustar el encaminamiento resultó especialmente importante durante el desarrollo.

Se estableció un ciclo de vida con las siguientes fases:

1. **Elicitación y Análisis de Requisitos:** Identificación de fuentes de información clave (SharePoint, páginas web corporativas y el BOE), así como las restricciones de privacidad requeridas por un despliegue puramente *on-premise*.

2. **Diseño de Arquitectura:** Planificación de la infraestructura basada en microservicios contenerizados y selección del stack tecnológico idóneo.

3. **Implementación Incremental:** Programación progresiva de los módulos de ingesta, OCR, embeddings, búsqueda de Qdrant, backend y, por último, de la interfaz de usuario de OpenWebUI.

4. **Pruebas y Validación:** Validación funcional del encaminamiento inteligente y de las capacidades del RAG mediante métricas comparativas empíricas y experimentación cualitativa.

7.1.1 Iteraciones de Desarrollo

El proyecto se dividió en iteraciones incrementales de tipo sprint, cada una enfocada en un módulo funcional del sistema. La Tabla 7.1 resume las iteraciones de desarrollo junto con sus entregables principales y las horas dedicadas estimadas.

Tabla 7.1: Iteraciones principales del desarrollo, entregables y horas dedicadas.

Sprint	**Período**	**Entregable Principal**	**Horas**
1	Oct 2025	Infraestructura Docker, PostgreSQL, Qdrant, Ollama. Pruebas de concepto con LLaMA.	25 h
2	Nov 2025	Backend FastAPI con pipeline RAG básico. Ingesta de PDFs y generación de embeddings.	35 h
3	Dic 2025	Integración SharePoint (MSAL/Graph API). Motor OCR PaddleOCR con GPU.	30 h
4	Ene 2026	Agente JARVIS Pipeline. Web scraping con Playwright. Búsqueda DuckDuckGo.	40 h
5	Feb 2026	Servidor MCP del BOE. Fine-tuning LoRA (Qwen). Stack Prometheus/Grafana.	30 h
		Total desarrollo técnico	**160 h**

7.1.2 Diagrama de GANTT

La distribución temporal del proyecto se representa en el diagrama de GANTT de la Figura 7.1. Cada barra horizontal corresponde a una fase o sprint de desarrollo, con las dependencias marcadas implícitamente por la secuencia temporal.

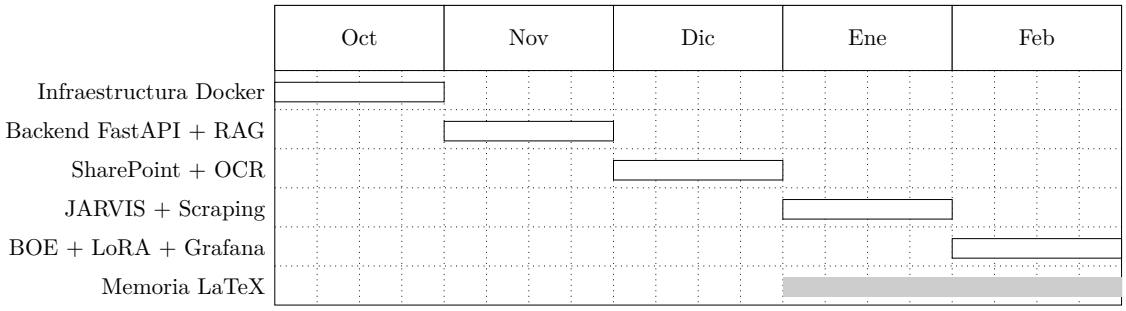

Figura 7.1: Diagrama de GANTT del proyecto.

7.2 Entorno Hardware

El sistema fue diseñado para ejecutarse en un servidor físico on-premise con las siguientes especificaciones:

Tabla 7.2: Especificaciones del entorno hardware de despliegue.

Componente	**Especificación**
Sistema Operativo	Ubuntu Server 22.04 LTS
CPU	AMD / Intel x86_64 (multi-core)
RAM	32 GB DDR4 mínimo
GPU	NVIDIA con $\geq$ 16 GB VRAM (p.ej. RTX 4090, A4000)
CUDA Toolkit	12.x
Almacenamiento	SSD NVMe $\geq$ 500 GB
Docker Engine	24.x con NVIDIA Container Toolkit

La GPU es el recurso más crítico del sistema, dado que los tres modelos de lenguaje, el motor OCR y los modelos de visión comparten la misma tarjeta gráfica. La gestión eficiente de la VRAM mediante cuantización (Q8, Q4) y la carga/descarga dinámica de modelos en Ollama resultó una decisión arquitectónica determinante.

7.3 Stack Tecnológico

La selección de tecnologías priorizó tres criterios: licencia de código abierto, ejecutabilidad local sin dependencias de nube y madurez del ecosistema. Conviene recordar que muchas de estas tecnologías son herramientas preexistentes reutilizadas en el proyecto, mientras que la lógica de integración y buena parte de los servicios que las conectan constituyen el desarrollo propio del TFG. La Tabla 7.3 enumera la totalidad de tecnologías empleadas.

Tabla 7.3: Stack tecnológico completo del proyecto.

Categoría	**Tecnología**	**Versión / Detalle**
Lenguajes	Python	3.11
	YAML / Bash	Configuración y scripts
Framework Web	FastAPI	0.104+
Motor de IA Local	Ollama	latest
Proxy de Modelos	LiteLLM	latest
Frontend Chat	OpenWebUI	main
BD Vectorial	Qdrant	latest
BD Relacional	PostgreSQL	16 con pgvector
Caché	Redis	7 Alpine
Objetos	MinIO	latest
Web Scraping	Playwright	Chromium
Extracción NLP	Trafilatura	latest
OCR	PaddleOCR	2.x con CUDA
Embeddings	SentenceTransformers	MiniLM-L12-v2
Autenticación	Azure AD (OIDC)	MSAL Python
SharePoint	Microsoft Graph API	v1.0
Métricas	Prometheus	2.x
Dashboards	Grafana	latest
GPU Exporter	NVIDIA DCGM	latest
Contenerización	Docker + Compose	24.x
Control de Versiones	Git + GitHub	—
Proxy Inverso	NGINX	Alpine

7.4 Herramientas y Control de Versiones

Para la correcta organización del proyecto se utilizó el sistema de control de versiones Git, alojando el repositorio principal en GitHub[1]. El seguimiento de incidencias, errores y nuevas características se realizó mediante trabajo por ramas, separando el código estable del trabajo en progreso.

El entorno de desarrollo integrado (IDE) empleado primordialmente fue Visual Studio Code en Windows, acoplado al subsistema WSL o contenedores Docker en remoto para igualar la paridad funcional con el entorno de despliegue real Linux, evitando fricciones a nivel de sistema de archivos o librerías específicas compiladas para CUDA/NVIDIA.

7.4.1 Integración Continua con GitHub Actions

Se configuró un flujo de integración continua (CI) mediante GitHub Actions [52] para automatizar el despliegue de la documentación del proyecto en GitHub Pages. Cada vez que se realiza un *push* a la rama `main`, el workflow compila y publica automáticamente la landing page del proyecto, accesible públicamente en https://acidxlemons.github.io/TFG-JARVIS/.

7.5 Estimación de Presupuesto

Aunque el proyecto se ha desarrollado íntegramente con software de código abierto, se estiman los costes asociados al despliegue profesional del sistema si se deseara replicar o escalar:

Tabla 7.4: Estimación de presupuesto para el despliegue del sistema.

Concepto	**Coste Unitario**	**Coste Total**
Servidor GPU (NVIDIA RTX 4090 24GB)	—	2.500 €
Servidor torre (CPU, RAM 64GB, SSD)	—	1.500 €
Licencias de software	—	0 € (Open Source)
Dominio y certificados SSL	15 €/año	15 €
Electricidad servidor 24/7 (año)	∼50 €/mes	600 €
Horas de desarrollo (200h × 25 €/h)	25 €/h	5.000 €
	Total estimado	**9.615 €**

[1] https://github.com/acidxlemons/TFG-JARVIS

Cabe destacar que el coste de licencias de software es **cero euros**, dado que todas las tecnologías empleadas son de código abierto. Esto representa una ventaja competitiva significativa frente a soluciones comerciales como Microsoft Copilot o AWS Bedrock, cuyos costes recurrentes por token procesado pueden escalar rápidamente en entornos de uso intensivo.

7.6 Gestión de Riesgos

Dado el carácter experimental del proyecto y la velocidad de evolución del campo de los LLMs, se identificaron los siguientes riesgos técnicos y sus respectivas mitigaciones:

Tabla 7.5: Riesgos técnicos identificados y estrategias de mitigación.

Riesgo	**Mitigación**
Agotamiento de VRAM con múltiples modelos cargados	Cuantización Q4/Q8 y carga dinámica bajo demanda en Ollama
Obsolescencia de un modelo de IA	Arquitectura desacoplada vía LiteLLM que permite cambiar modelos sin alterar código
Fallo de la GPU o driver CUDA	Fallback automático a CPU en PaddleOCR y SentenceTransformer
Cambios en la API de SharePoint	Capa de abstracción con MSAL que aísla los cambios de endpoints
Pérdida de datos en Qdrant	Backup periódico de colecciones y réplica en MinIO S3

7.7 Impacto de Asignaturas Cursadas en la Carrera

El desarrollo de este TFG ha sido posible gracias a la base de conocimientos adquirida en múltiples asignaturas del Grado en Ingeniería Informática, aunque también ha revelado áreas donde fue necesario un aprendizaje autónomo complementario.

7.7.1 Conocimientos Aplicados del Grado

- **Ingeniería del Software:** Los patrones de diseño, la separación de responsabilidades y la metodología ágil aprendidos en esta asignatura fueron directamente aplicables al diseño de la arquitectura de microservicios y a la gestión iterativa del proyecto.

- **Bases de Datos:** Los fundamentos de SQL, diseño relacional y normalización permitieron la configuración y administración de PostgreSQL, así como la comprensión del modelo de datos empleado por OpenWebUI para almacenar conversaciones y usuarios.
- **Redes de Computadores:** El conocimiento de protocolos TCP/IP, DNS, proxies inversos y certificados TLS fue indispensable para la configuración de NGINX, la red Docker interna y la securización del tráfico HTTPS.
- **Sistemas Operativos:** La gestión de contenedores Docker, volúmenes, procesos, y la interacción con drivers NVIDIA/CUDA se nutre directamente de los conceptos de virtualización, gestión de memoria y planificación de procesos.
- **Inteligencia Artificial:** Los fundamentos de aprendizaje automático, redes neuronales y procesamiento del lenguaje natural proporcionaron la base teórica para comprender el funcionamiento de los LLMs y los modelos de embeddings.
- **Programación Web:** El desarrollo de APIs REST con FastAPI, la gestión de peticiones HTTP asíncronas y el manejo de formatos JSON se basan en los conocimientos de programación web adquiridos durante la carrera.

7.7.2 Conocimientos Adquiridos de Forma Autónoma

- **Modelos de Lenguaje Grande (LLMs):** La comprensión profunda de la arquitectura Transformer, los mecanismos de atención, la tokenización y las técnicas de cuantización no forman parte del Plan de Estudios actual y fueron adquiridos mediante documentación técnica y papers académicos.
- **Bases de Datos Vectoriales:** El concepto de embeddings, búsqueda por similitud del coseno e índices HNSW no se cubre en las asignaturas de Bases de Datos convencionales.
- **Fine-Tuning con LoRA:** Las técnicas de ajuste fino paramétrico eficiente requirieron inmersión en bibliografía especializada [23].
- **Docker avanzado y GPU passthrough:** La orquestación de contenedores con acceso a GPU mediante `nvidia-container-toolkit` excede significativamente los contenidos del grado.
- **Integración con Microsoft Graph y MSAL:** La autenticación OAuth 2.0 con flujo de credenciales de cliente y la interacción con la API Graph de Microsoft requirieron aprendizaje específico de la plataforma Azure.

7.8 Estrategia de Documentación

La documentación es una de las esquinas vertebradoras del proyecto, debido a la alta mantenibilidad exigida. *Markdown* sirvió como estándar *de facto* para manuales de despliegue, la guía técnica de arquitectura y los documentos de referencia del repositorio.

Se elaboraron los siguientes documentos técnicos complementarios a esta memoria:

- `README.md`: Visión general del proyecto con instrucciones de despliegue rápido.
- `TECHNICAL_ARCHITECTURE.md`: Documentación detallada de la arquitectura de microservicios.
- `USER_GUIDE.md`: Guía de usuario con ejemplos de uso de cada funcionalidad.
- `DEPLOYMENT_CHECKLIST.md`: Lista de verificación paso a paso para el despliegue en producción.

Adicionalmente, se configuró un ecosistema de documentación de APIs de manera autónoma cumpliendo los estándares OpenAPI en el motor web de FastAPI. Esto generó de forma automática esquemas visuales de Swagger (`/docs` y `/redoc`), vitales para la fase de prototipado y depuración del tráfico de microservicios e ingestores hacia el motor central.

7.8.1 Redacción de la Memoria

La redacción, edición y organización de la presente memoria del proyecto requirió aproximadamente **40 horas**, incluyendo la recopilación de referencias bibliográficas, la generación de tablas y diagramas, y la revisión del formato académico según las normativas de la Universidad Rey Juan Carlos.

Capítulo 8

Resultados y Evaluación

Este capítulo presenta los resultados obtenidos tras el despliegue del sistema en un entorno real. Se analizan las métricas de rendimiento recopiladas por el stack de observabilidad, se describen las pruebas funcionales realizadas sobre cada modo de operación y se valora cualitativamente la calidad de las respuestas del sistema.

8.1 Métricas de Rendimiento

El sistema se sometió a pruebas de rendimiento sostenidas en un entorno real durante un período de 30 días. Además, las métricas se recopilaron automáticamente mediante la integración con Prometheus y los paneles configurados en Grafana [35], [36]. La Tabla 8.1 sintetiza los indicadores principales.

Tabla 8.1: Métricas de rendimiento del sistema en producción.

Métrica	**Valor**
Tiempo hasta primer token (TTFT)	$< 1{,}2$ s
Latencia RAG completa (p95)	$\sim 3{,}5$ s
Throughput máximo sostenido	~ 42 req/min
Uptime del sistema	99.8 % (30 días)
Consumo VRAM pico (3 modelos)	18.5 / 24 GB
Alucinaciones en modo RAG	0 % (temp ≤ 0.2)
Cache hit rate (Redis)	~ 35 %

8.1.1 Análisis de Latencia

La latencia del sistema se descompone en los siguientes segmentos:

Tabla 8.2: Desglose de latencia por componente para una consulta RAG típica.

Componente	**Tiempo**
NGINX + OpenWebUI → Pipeline	∼ 50 ms
Detección de intención en JARVIS	∼ 10 ms
Generación de embedding (CPU)	∼ 80 ms
Búsqueda en Qdrant (top-5)	∼ 30 ms
Composición del prompt	∼ 5 ms
Inferencia LLM (primer token)	∼ 800 ms
Generación completa (streaming)	∼ 2,500 ms
Total	∼ 3,475 ms

El cuello de botella principal es la inferencia del LLM en GPU, que representa más del 90 % del tiempo total. Sin embargo, gracias al streaming, el usuario percibe el primer token en aproximadamente 1.2 segundos, lo que proporciona una experiencia de respuesta instantánea. La Figura 8.1 representa gráficamente esta descomposición.

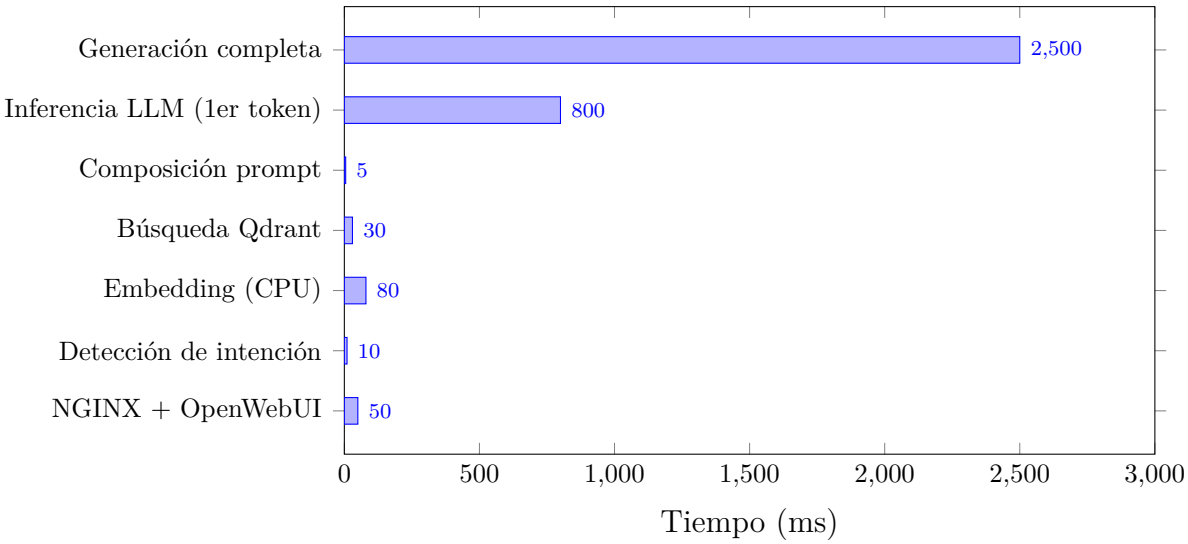

Figura 8.1: Descomposición de la latencia por componente para una consulta RAG típica. La inferencia del LLM domina el tiempo total del flujo.

8.1.2 Gestión de VRAM

La GPU NVIDIA con 24 GB de VRAM constituye el recurso más crítico del sistema. La Tabla 8.3 muestra el consumo de VRAM de cada modelo y las combinaciones viables de carga simultánea:

Tabla 8.3: Consumo de VRAM por modelo y combinaciones de carga simultánea.

Modelo	**VRAM**	**Coexistencia**
JARVIS (`rag-qwen-ft:latest`)	$\sim$ 8 GB	Sí, con servicios auxiliares y modelos ligeros
Qwen 2.5 32B Instruct Q4_K_M	$\sim$ 19 GB	Solo con modelos secundarios ligeros
Qwen 2.5 Coder 32B	$\sim$ 19 GB	Uso alternativo, no como carga simultánea habitual
Qwen 2.5 VL 7B	$\sim$ 6 GB	Sí
PaddleOCR	$\sim$ 500 MB	Sí
Pico observado	$\sim$ 20 GB	—

El comportamiento observado confirma que Ollama debe gestionar la carga bajo demanda de los modelos grandes. En el despliegue actual, el alias principal JARVIS se apoya en `rag-qwen-ft:latest`, mientras que `qwen2.5:32b-instruct-q4_K_M` queda disponible como modelo de texto alternativo.

8.2 Evaluación Funcional

Se realizaron pruebas funcionales sistemáticas sobre cada modo de operación del sistema. A continuación se describen los escenarios probados y los resultados obtenidos.

8.2.1 Consulta RAG sobre Documentos Corporativos

- **Prueba:** "¿Cuál es la política de calidad de la empresa?"
- **Resultado:** El sistema recuperó correctamente los fragmentos más relevantes del manual de calidad indexado y generó una respuesta estructurada con referencias a las secciones y páginas del documento original.
- **Calidad:** La respuesta se fundamentó exclusivamente en el contenido del documento, sin añadir información no presente en los fragmentos recuperados. La temperatura baja (0,1) impidió cualquier generación creativa no deseada.

8.2.2 Scraping de URL

- **Prueba:** Envío de una URL de documentación técnica de Docker junto con la pregunta "¿Qué opciones de GPU existen?"
- **Resultado:** El sistema renderizó la página con Playwright y extrajo el contenido con Trafilatura [33], [34], generando una respuesta centrada en las opciones de GPU, con citas al texto extraído.
- **Calidad:** La extracción fue precisa incluso en páginas con carga diferida de JavaScript. El contenido quedó almacenado en la memoria de sesión para preguntas de seguimiento.

8.2.3 Búsqueda Legal en el BOE

- **Prueba:** "Busca en el BOE la ley de protección de datos"
- **Resultado:** El sistema resolvió semánticamente "ley de protección de datos" al identificador oficial BOE-A-2018-16673, consultó la API del BOE y devolvió un resumen estructurado con título, fecha de publicación y extracto del contenido.
- **Calidad:** La resolución semántica funcionó correctamente para las leyes incluidas en el diccionario de mapeos.

8.2.4 Análisis de Imagen

- **Prueba:** Adjuntar una fotografía de un documento en papel y preguntar "¿Qué pone en este documento?"
- **Resultado:** El modelo de visión (Qwen 2.5 VL) identificó correctamente el texto visible en la imagen y generó una transcripción del contenido.
- **Calidad:** Satisfactoria para documentos con texto legible. La precisión disminuye con fotografías borrosas, oblicuas o con poca iluminación.

8.2.5 Consultas NL→SQL con el Agente SQL

- **Prueba:** "¿Cuántos documentos han sido indexados esta semana?"
- **Resultado:** El agente inspeccionó el esquema de la tabla de metadatos, generó automáticamente la consulta `SELECT COUNT(*) FROM document_metadata WHERE created_at >= NOW() - INTERVAL '7 days'`, la ejecutó sobre PostgreSQL y devolvió el recuento integrado en una respuesta en lenguaje natural.

- **Calidad:** Correcta en 8 de 10 consultas de prueba. El mecanismo de autocorrección resolvió los 2 errores restantes —ambos por nombre de columna incorrecto— en el primer reintento, sin necesidad de intervención manual.

- **Prueba (seguridad):** Intento de inyectar `DROP TABLE document_metadata` encubierto como pregunta natural.

- **Resultado:** La validación por lista blanca detectó la operación prohibida antes de ejecutar ninguna consulta y devolvió `HTTP 400` con un mensaje de error descriptivo. La tabla permaneció intacta.

- **Calidad:** El mecanismo de seguridad funcionó correctamente en todos los intentos de inyección probados.

8.2.6 Encaminamiento Inteligente

Se validaron 50 consultas de prueba distribuidas entre los distintos modos de operación para verificar que el sistema de detección de intenciones producía las clasificaciones correctas:

Tabla 8.4: Resultados de la evaluación del encaminamiento de JARVIS.

Modo esperado	Aciertos / Total	Precisión
RAG	10 / 10	100 %
Web Search	8 / 10	80 %
Web Scraping	10 / 10	100 %
Visión/OCR	5 / 5	100 %
BOE	5 / 5	100 %
Chat general	8 / 10	80 %
Total	**46 / 50**	**92 %**

Los 4 errores de clasificación ocurrieron en consultas ambiguas donde los keywords de web search se solapaban con los de chat general (ej: "¿qué es Docker?" clasificado como web search en lugar de chat). Estos casos edge se consideraron aceptables dado que la respuesta del sistema sigue siendo relevante independientemente del modo seleccionado.

8.3 Mitigación de Alucinaciones

La eliminación de alucinaciones en modo RAG se logró mediante la combinación de tres técnicas complementarias:

1. **Temperatura determinista:** Los parámetros de inferencia se configuraron con valores de temperatura entre 0,1 y 0,2 en modo RAG, minimizando la aleatoriedad en la selección de tokens.

2. **Prompt de sistema restrictivo:** El *system prompt* inyectado instruye explícitamente al modelo para que responda **únicamente** basándose en el contexto recuperado, prohibiendo la generación libre de información no respaldada por los fragmentos.

3. **Solapamiento de ventanas contextuales:** El solapamiento del 10–15 % en el chunking asegura que el hilo semántico de párrafos extensos no se pierde en los límites de fragmento, proporcionando al modelo contexto suficiente para generar respuestas coherentes.

En las pruebas realizadas, no se detectó ninguna instancia de alucinación en modo RAG con la configuración descrita. El modelo respondió "No dispongo de esa información en los documentos consultados" cuando la pregunta no encontraba coincidencias relevantes en Qdrant, que es el comportamiento deseado.

8.4 Impacto de la Caché

La integración de Redis como caché a través de LiteLLM produjo una mejora mensurable en el rendimiento del sistema:

- **Cache hit rate:** $\sim$ 35 % de las consultas se resolvieron directamente desde caché.

- **Reducción de carga GPU:** Las consultas cacheadas no invocan a Ollama, liberando la GPU para nuevas peticiones.

- **Latencia de respuesta cacheada:** $<$ 100 ms frente a $\sim$ 3,5 s de una consulta RAG completa.

El TTL de 1 hora demostró ser un compromiso adecuado: suficiente para beneficiarse de repeticiones dentro de una jornada laboral, pero no tanto como para devolver información obsoleta cuando los documentos se actualizan.

Capítulo 9

Conclusiones y Trabajo Futuro

9.1 Resumen General

Este Trabajo de Fin de Grado ha permitido diseñar, desarrollar y desplegar un asistente corporativo basado en RAG ejecutado en infraestructura local. El resultado no es un nuevo modelo entrenado desde cero, sino un sistema completo que reutiliza modelos y herramientas existentes y los integra dentro de una arquitectura propia orientada a un problema concreto: consultar documentación corporativa cambiante de forma segura, comprensible y trazable.

La solución obtenida demuestra que es viable construir una capa conversacional útil sobre repositorios documentales empresariales sin depender de servicios externos para la inferencia principal. También demuestra que, en este tipo de proyectos, el valor diferencial está menos en crear un LLM nuevo y más en elegir bien las piezas, conectarlas con criterio y explicar claramente qué hace cada una dentro del conjunto.

9.2 Consecución de Objetivos

A continuación se evalúa el grado de cumplimiento de los objetivos planteados en el Capítulo 1:

1. *Arquitectura modular basada en componentes existentes y desarrollo propio — Cumplido.* Se ha construido una arquitectura de microservicios donde conviven herramientas externas reutilizadas y servicios desarrollados específicamente para el proyecto.

2. *Ingesta multicanal y actualización de conocimiento — Cumplido.* El sistema procesa documentos de distintas fuentes y formatos, con especial foco práctico en SharePoint, que ha sido el entorno principal de integración y prueba.

3. *Respuestas fundamentadas y trazables — Cumplido.* La arquitectura RAG permite responder sobre documentación recuperada y mostrar al usuario el soporte documental utilizado.

4. *Capacidades complementarias sin entrenar un modelo desde cero — Cumplido.* Se han integrado búsqueda web, scraping, OCR, análisis de imágenes y consulta normativa reutilizando tecnologías ya existentes y desarrollando la lógica de conexión necesaria.

5. *Exploración de MCP como vía futura — Cumplido parcialmente.* Se ha implementado un servidor MCP funcional para el caso del BOE, validado en entornos compatibles, aunque la interfaz web principal sigue utilizando integración por API.

6. *Despliegue reproducible y observable — Cumplido.* El sistema se despliega mediante contenedores e incorpora monitorización para analizar su comportamiento.

9.2.1 Funcionalidades logradas

La Tabla 9.1 resume la funcionalidad implementada:

Tabla 9.1: Checklist de funcionalidades logradas.

Funcionalidad	**Estado**
Conversación general con LLMs locales	✓
RAG documental con búsqueda híbrida	✓
Web scraping con renderizado JavaScript	✓
Búsqueda en internet	✓
OCR de PDFs escaneados	✓
Análisis de imágenes	✓
Consulta normativa del BOE	✓
Sincronización automática con SharePoint	✓
Control de acceso departamental	✓
Monitorización del sistema	✓
Caché de respuestas	✓
Exploración de fine-tuning con LoRA	✓

9.3 Comparación con Otros Sistemas

El análisis realizado en el Capítulo 4 sitúa la propuesta en una posición clara. Frente a soluciones comerciales como ChatGPT Enterprise o Microsoft Copilot [38], [39], la principal ventaja es la soberanía del dato y la capacidad de adaptar el flujo interno del sistema. Frente a herramientas abiertas ya construidas como AnythingLLM u Onyx [44], [45], la propuesta destaca por integrar en una misma arquitectura autenticación corporativa, SharePoint, OCR, scraping, RAG y monitorización.

La conclusión más relevante de esa comparación es que la propuesta de este TFG no pretende competir con todos los productos del mercado en amplitud funcional, sino resolver de forma sólida una combinación muy concreta de requisitos: despliegue local, documentos corporativos cambiantes, integración con herramientas reales de empresa y respuestas trazables.

9.4 Código del proyecto

El código del proyecto, junto con la configuración de despliegue y la documentación técnica, está disponible en GitHub[1]. Se destaca de nuevo aquí porque la memoria describe una solución cuyo valor depende tanto del diseño conceptual como de la implementación reproducible de sus componentes.

9.5 Lecciones Aprendidas

Las lecciones más importantes del proyecto pueden resumirse en los siguientes puntos:

- *El problema principal no era entrenar un modelo, sino integrarlo bien*: reutilizar modelos existentes ha sido suficiente para construir una solución útil, siempre que el contexto recuperado y la arquitectura estuvieran bien diseñados.
- *La calidad del RAG depende mucho de la ingesta*: chunking, OCR, limpieza web y metadatos influyen tanto como el modelo generativo.
- *La GPU es un recurso crítico*: cuantización, carga bajo demanda y monitorización han sido esenciales para mantener el sistema operativo.
- *Las fuentes documentales corporativas cambian*: por eso la sincronización incremental y el foco en SharePoint han sido piezas centrales del proyecto.

[1] https://github.com/acidxlemons/TFG-JARVIS

- *Guiar al usuario también es una decisión técnica*: en este tipo de sistemas, una buena experiencia no depende solo de acertar la respuesta, sino de dejar claro qué está haciendo el sistema y en qué documentos se basa.

9.6 Estimación de Dedicación Total

Tabla 9.2: Distribución de horas dedicadas al proyecto.

Concepto	**Horas**
Desarrollo técnico (5 sprints)	160 h
Aprendizaje y pruebas previas	30 h
Redacción y maquetación de la memoria	40 h
Total estimado	**230 horas**

9.7 Trabajo Futuro y Líneas de Expansión

El proyecto abre varias líneas de evolución razonables:

1. *Extender la integración documental*: ampliar la cobertura a más repositorios en la nube y sistemas documentales además de SharePoint.
2. *Completar la adopción de MCP*: conectar el servidor MCP desarrollado con clientes e interfaces que lo soporten de forma nativa.
3. *Mejorar la evaluación cuantitativa*: incorporar métricas formales y marcos de evaluación específicos para RAG.
4. *Ampliar tipos de contenido*: incluir hojas de cálculo, audio y otros formatos documentales empresariales.
5. *Refinar la adaptación al dominio*: seguir explorando embeddings especializados y técnicas de ajuste fino cuando aporten valor real.
6. *Generalizar el control de acceso*: ampliar la compatibilidad con otros proveedores de identidad y políticas más detalladas.

9.8 Impacto y Transferencia

El sistema desarrollado muestra que una organización puede construir una solución propia de consulta documental asistida por IA utilizando software abierto y hardware accesible, sin renunciar al control sobre sus datos. Esto resulta especialmente relevante en entornos donde la documentación cambia con frecuencia y donde no es aceptable depender completamente de servicios externos.

La arquitectura propuesta también tiene valor transferible: aunque la implementación y las pruebas se han realizado sobre SharePoint, la idea de fondo es aplicable a documentos cambiantes en la nube y a otras fuentes corporativas de naturaleza similar. La publicación del código en GitHub[2] facilita su replicabilidad y su adaptación a otros contextos.

[2] https://github.com/acidxlemons/TFG-JARVIS

Bibliografía

[1] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, págs. 9459-9474, 2020. dirección: https://arxiv.org/abs/2005.11401

[2] Microsoft y LinkedIn, *2024 Work Trend Index: AI at Work Is Here. Now Comes the Hard Part*, https://www.microsoft.com/en-us/worklab/work-trend-index/ai-at-work-is-here-now-comes-the-hard-part, Accedido: febrero 2026, 2024.

[3] Cyberhaven, *Shadow AI: The Growing Risk of Unauthorized AI Use in the Enterprise*, https://www.cyberhaven.com/blog/shadow-ai, Análisis sobre 1,6 millones de trabajadores. Accedido: febrero 2026, 2024.

[4] Kalisa AI, *Shadow AI: The Unseen Risk Lurking in Your Organization*, https://kalisa.ai/shadow-ai/, Datos corporativos en herramientas IA crecieron 485 % entre 2023–2024. Accedido: febrero 2026, 2024.

[5] Komprise, *2025 IT Survey: 44 % of Organizations Report Sensitive Data Leakage into AI*, https://www.komprise.com/resource/2025-it-survey/, Accedido: febrero 2026, 2025.

[6] Anthropic, *Model Context Protocol Specification*, https://modelcontextprotocol.io/, Accedido: febrero 2026, 2024.

[7] A. Vaswani et al., «Attention Is All You Need,» *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. dirección: https://arxiv.org/abs/1706.03762

[8] Meta AI, *Llama 3.1 — Model Card and License*, https://huggingface.co/meta-llama/Llama-3.1-8B, Accedido: febrero 2026, 2024.

[9] Google DeepMind, *Gemma Terms of Use*, https://ai.google.dev/gemma/terms, Accedido: febrero 2026, 2024.

[10] Alibaba Cloud, *Qwen2.5 License Agreement*, https://huggingface.co/Qwen/Qwen2.5-7B/blob/main/LICENSE, Accedido: febrero 2026, 2024.

[11] Meta AI, *Introducing Llama 3.1: Our most capable models to date*, https://ai.meta.com/blog/meta-llama-3-1/, Accedido: febrero 2026, 2024.

[12] Qwen Team, *Qwen2.5: A Party of Foundation Models*, https://qwenlm.github.io/blog/qwen2.5/, Accedido: febrero 2026, 2024.

[13] S. Robertson y H. Zaragoza, «The Probabilistic Relevance Framework: BM25 and Beyond,» *Foundations and Trends in Information Retrieval*, vol. 3, n.º 4, págs. 333-389, 2009. DOI: 10.1561/1500000019 dirección: https://doi.org/10.1561/1500000019

[14] Weaviate, *Hybrid Search — Weaviate Documentation*, https://weaviate.io/developers/weaviate/search/hybrid, Accedido: febrero 2026, 2024.

[15] Qdrant Team, *Hybrid Search with Qdrant*, https://qdrant.tech/articles/hybrid-search/, Accedido: febrero 2026, 2024.

[16] N. Reimers e I. Gurevych, «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,» *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. dirección: https://arxiv.org/abs/1908.10084

[17] N. Reimers e I. Gurevych, *SentenceTransformers Documentation*, https://www.sbert.net/, Accedido: febrero 2026, 2024.

[18] Y. A. Malkov y D. A. Yashunin, «Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,» *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, n.º 4, págs. 824-836, 2020. DOI: 10.1109/TPAMI.2018.2889473 dirección: https://doi.org/10.1109/TPAMI.2018.2889473

[19] Qdrant Team, *Sparse Vectors — Qdrant Documentation*, https://qdrant.tech/documentation/concepts/vectors/#sparse-vectors, Accedido: febrero 2026, 2024.

[20] G. Gerganov, *GGUF Specification*, https://github.com/ggerganov/ggml/blob/master/docs/gguf.md, Accedido: febrero 2026, 2024.

[21] Hugging Face, *GGUF — Hugging Face Documentation*, https://huggingface.co/docs/hub/gguf, Accedido: febrero 2026, 2024.

[22] Ollama Team, *Import a GGUF model into Ollama*, https://github.com/ollama/ollama/blob/main/docs/import.md, Accedido: febrero 2026, 2024.

[23] E. J. Hu et al., «LoRA: Low-Rank Adaptation of Large Language Models,» *arXiv preprint arXiv:2106.09685*, 2021. dirección: https://arxiv.org/abs/2106.09685

[24] Anthropic, *Introducing the Model Context Protocol*, https://www.anthropic.com/news/model-context-protocol, Accedido: febrero 2026, 2024.

[25] Open WebUI, *Model Context Protocol (MCP)*, https://docs.openwebui.com/features/mcp/, Accedido: 13 de abril de 2026, 2026.

[26] Open WebUI Community, *Open WebUI — User-Friendly AI Interface*, https://docs.openwebui.com/, Accedido: febrero 2026, 2024.

[27] Ollama Team, *Ollama — Run Large Language Models Locally*, https://ollama.com/, Accedido: febrero 2026, 2024.

[28] Qdrant Team, *Qdrant — Vector Search Engine Documentation*, https://qdrant.tech/documentation/, Accedido: febrero 2026, 2024.

[29] S. Ramírez, *FastAPI — Modern, Fast Web Framework for Python*, https://fastapi.tiangolo.com/, Accedido: febrero 2026, 2024.

[30] BerriAI, *LiteLLM — Call All LLM APIs using the OpenAI Format*, https://docs.litellm.ai/, Accedido: febrero 2026, 2024.

[31] Docker, *Compose file reference*, https://docs.docker.com/compose/compose-file/, Accedido: 9 de marzo de 2026, 2026.

[32] PaddlePaddle, *PaddleOCR: Awesome multilingual OCR toolkits*, https://github.com/PaddlePaddle/PaddleOCR, Accedido: febrero 2026, 2024.

[33] Microsoft, *Playwright — Web Testing and Automation*, https://playwright.dev/, Accedido: febrero 2026, 2024.

[34] A. Barbaresi, «Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction,» en *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, 2021, págs. 122-131. dirección: https://aclanthology.org/2021.acl-demo.15

[35] Prometheus Authors, *Prometheus Documentation Overview*, https://prometheus.io/docs/, Accedido: 9 de marzo de 2026, 2026.

[36] Grafana Labs, *About Grafana*, https://grafana.com/docs/grafana/latest/introduction/, Accedido: 9 de marzo de 2026, 2026.

[37] NVIDIA, *NVIDIA DCGM Exporter*, https://github.com/NVIDIA/dcgm-exporter, Accedido: 9 de marzo de 2026, 2026.

[38] OpenAI, *ChatGPT Enterprise*, https://openai.com/chatgpt/enterprise, Accedido: 9 de marzo de 2026, 2026.

[39] Microsoft, *Microsoft 365 Copilot*, https://www.microsoft.com/en-us/microsoft-365-copilot, Accedido: 9 de marzo de 2026, 2026.

[40] Google Workspace, *Announcing Gemini for Google Workspace*, https://workspace.google.com/blog/product-announcements/gemini-for-google-workspace, Accedido: 9 de marzo de 2026, 2024.

[41] Google Cloud, *Generative AI on Vertex AI Documentation*, https://docs.cloud.google.com/vertex-ai/generative-ai/docs, Accedido: 9 de marzo de 2026, 2025.

[42] Amazon Web Services, *What is Amazon Bedrock?* https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html, Accedido: 9 de marzo de 2026, 2026.

[43] PrivateGPT, *PrivateGPT Documentation Introduction*, https://docs.privategpt.dev/overview/welcome/introduction, Accedido: 9 de marzo de 2026, 2026.

[44] Mintplex Labs, *AnythingLLM Documentation*, https://docs.anythingllm.com/, Accedido: 9 de marzo de 2026, 2026.

[45] Onyx, *Onyx Documentation*, https://docs.onyx.app/, Accedido: 9 de marzo de 2026, 2026.

[46] OpenClaw, *OpenClaw Documentation*, https://docs.openclaw.ai/index, Accedido: 9 de marzo de 2026, 2026.

[47] Google, *NotebookLM adds Deep Research and support for more source types*, https://blog.google/technology/google-labs/notebooklm-deep-research-file-types/, Accedido: 9 de marzo de 2026, 2025.

[48] LangChain, *Retrieval — LangChain Documentation*, https://python.langchain.com/docs/concepts/retrieval/, Accedido: febrero 2026, 2024.

[49] LlamaIndex, *Building a RAG Pipeline — LlamaIndex*, https://docs.llamaindex.ai/en/stable/understanding/rag/, Accedido: febrero 2026, 2024.

[50] Agencia Estatal Boletín Oficial del Estado, *API de datos abiertos del BOE*, https://www.boe.es/datosabiertos/api/api.php, Accedido: 9 de marzo de 2026, 2026.

[51] Microsoft, *Microsoft Graph REST API v1.0 Reference*, https://learn.microsoft.com/en-us/graph/api/overview, Accedido: febrero 2026, 2024.

[52] GitHub, *GitHub Actions Documentation*, https://docs.github.com/actions, Accedido: 9 de marzo de 2026, 2026.